

神经网络基础



时间：2025年5月

TRANSWARP
星环科技



目录 >

CONTENTS

- 1 神经元
- 2 前馈神经网络
- 3 参数学习
- 4 Numpy手动实现神经网络



1

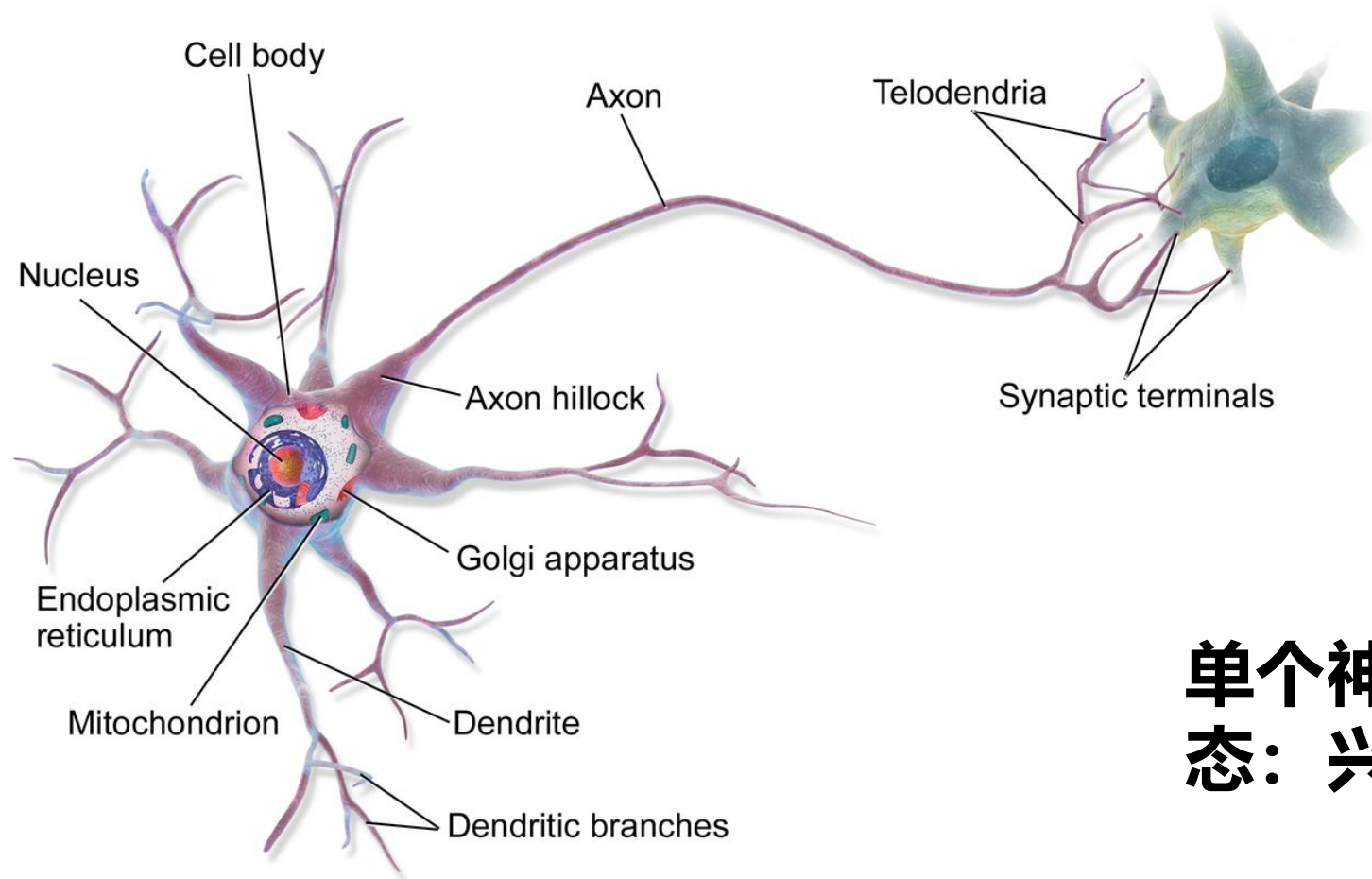
chapter

神经元

神经网络最早是作为一种主要的**连接主义**模型

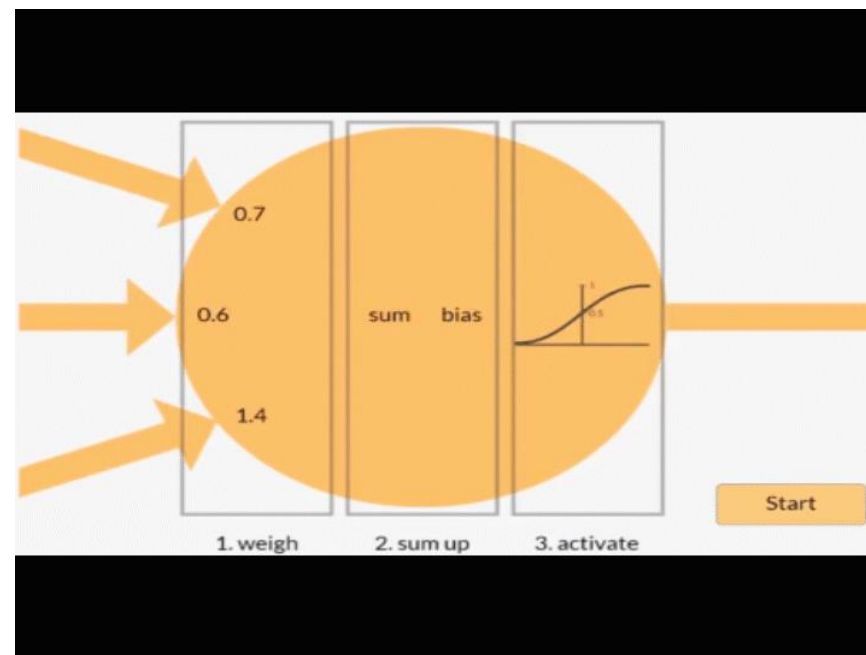
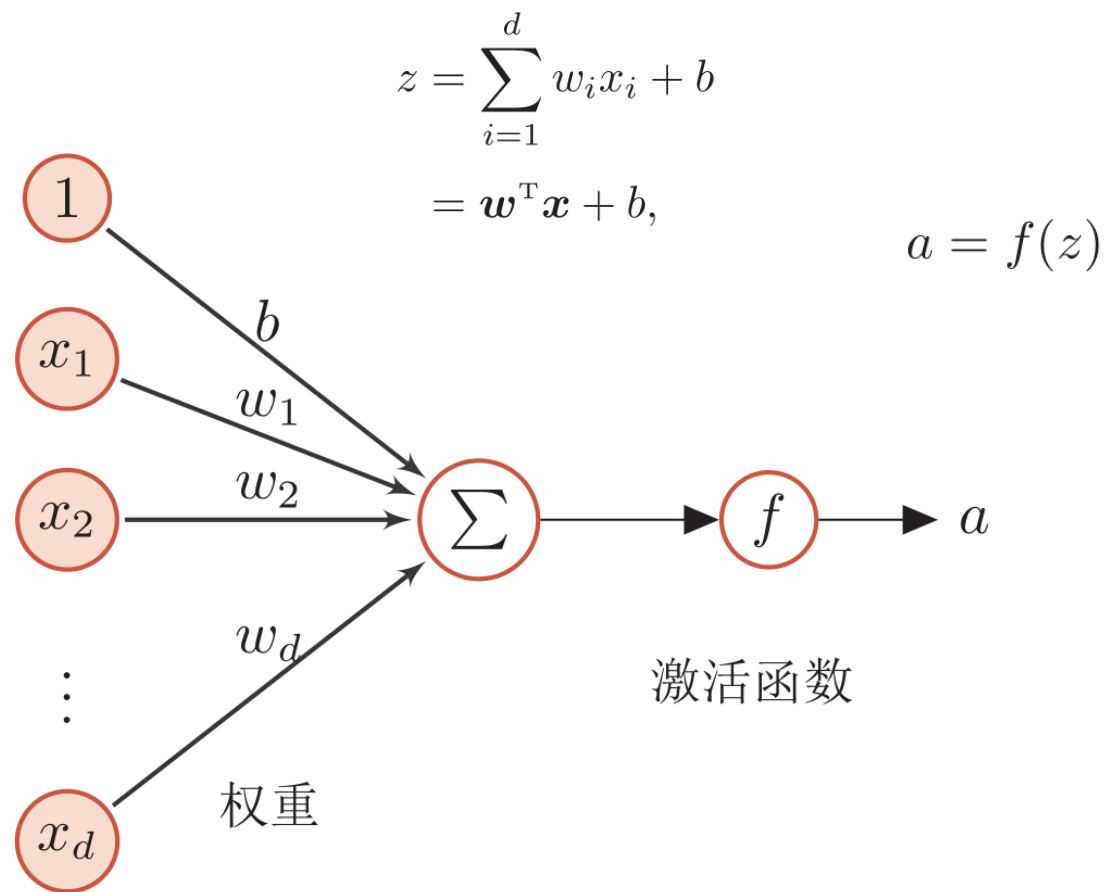
- 20世纪80年代后期，最流行的一种连接主义模型是分布式并行处理（Parallel Distributed Processing, PDP）网络，其有3个主要特性：
 - 1) 信息表示是分布式的（非局部的）；
 - 2) 记忆和知识是存储在单元之间的连接上；
 - 3) 通过逐渐改变单元之间的连接强度来学习新的知识。
- 引入误差**反向传播**来改进其学习能力之后，神经网络也越来越多地应用在各种机器学习任务上。

生物神经元



单个神经细胞只有两种状态：兴奋和抑制

人工神经元



激活函数的性质

- **连续并可导**（允许少数点上不可导）的**非线性函数**。
 - 可导的激活函数可以直接利用数值优化的方法来学习网络参数。
- 激活函数及其**导函数**要尽可能的**简单**
 - 有利于提高网络计算效率。
- 激活函数的导函数的**值域**要在一个合适的区间内
 - 不能太大也不能太小，否则会影响训练的效率和稳定性。
- **单调递增**

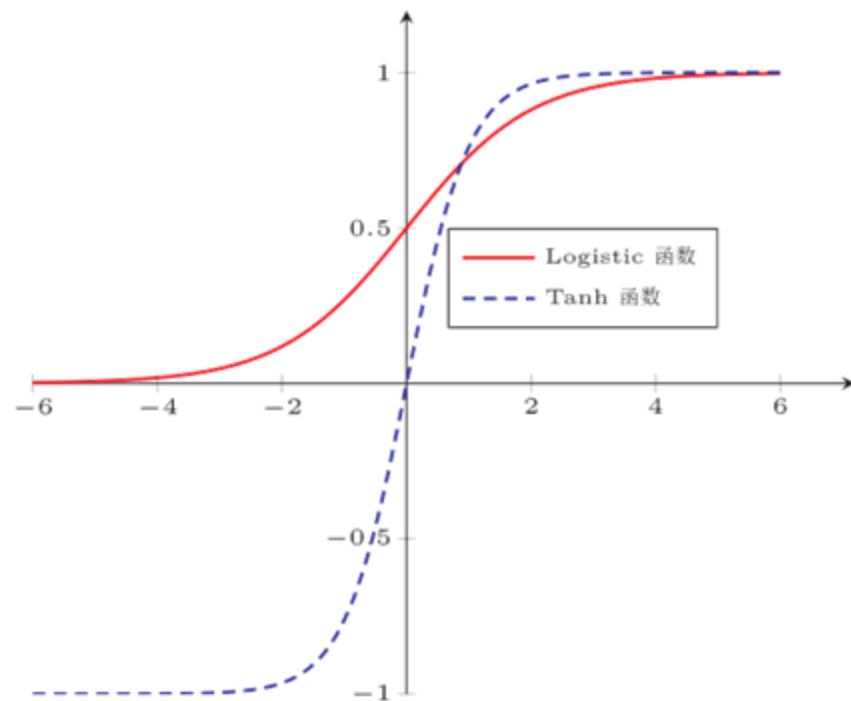
常见激活函数

性质:

- 容易**梯度消失**和**梯度爆炸**
- Tanh函数是零中心化的, 而sigmoid函数的输出恒大于0

$$\sigma(x) = \frac{1}{1 + \exp(-x)}$$

$$\tanh(x) = \frac{\exp(x) - \exp(-x)}{\exp(x) + \exp(-x)}$$



非零中心化的输出会使得其后的神经元的输入发生偏置偏移 (bias shift), 并进一步使得梯度下降的收敛速度变慢。

常见激活函数

$$\text{ReLU}(x) = \begin{cases} x & x \geq 0 \\ 0 & x < 0 \end{cases}$$

$$= \max(0, x).$$

$$\text{LeakyReLU}(x) = \begin{cases} x & \text{if } x > 0 \\ \gamma x & \text{if } x \leq 0 \end{cases}$$

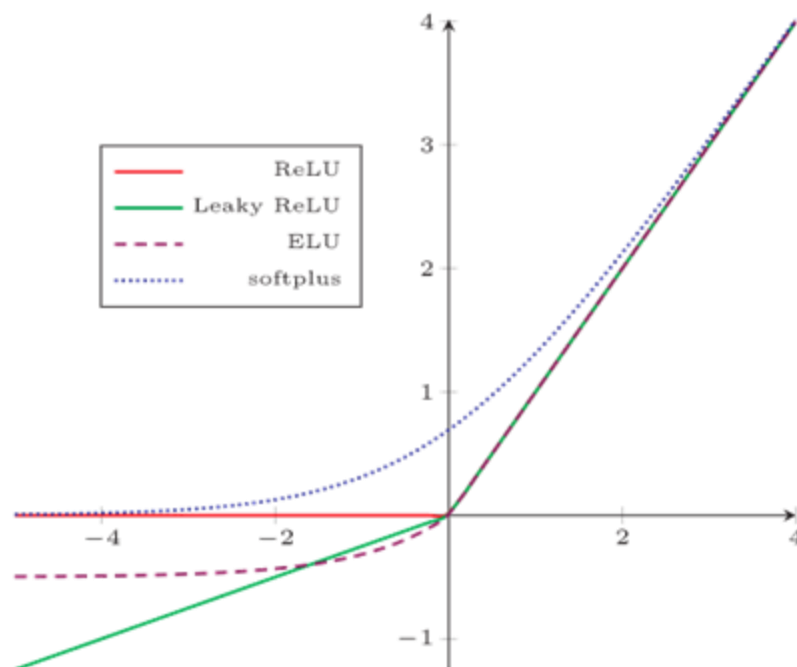
$$\text{PReLU}_i(x) = \begin{cases} x & \text{if } x > 0 \\ \gamma_i x & \text{if } x \leq 0 \end{cases}$$

$$= \max(0, x) + \gamma_i \min(0, x)$$

$$\text{ELU}(x) = \begin{cases} x & \text{if } x > 0 \\ \gamma(\exp(x) - 1) & \text{if } x \leq 0 \end{cases}$$

$$= \max(0, x) + \min(0, \gamma(\exp(x) - 1))$$

$$\text{softplus}(x) = \log(1 + \exp(x))$$

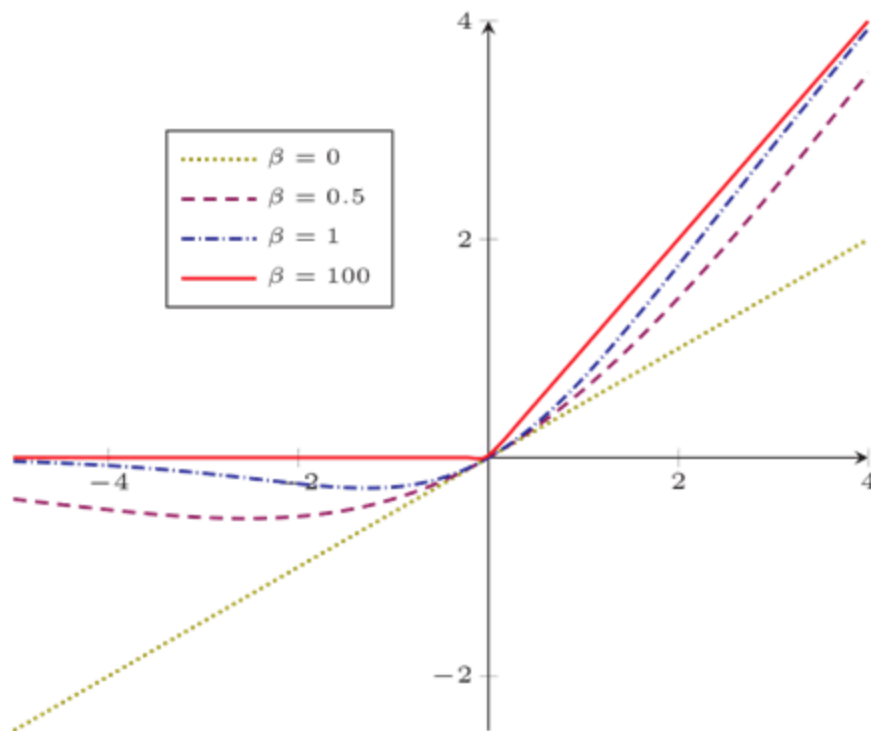


- ▶ 计算上更加高效
- ▶ 生物学合理性
 - ▶ 单侧抑制、宽兴奋边界
- ▶ 在一定程度上缓解梯度消失问题

死亡ReLU问题 (Dying ReLU Problem)

常见激活函数

Swish函数 $\text{swish}(x) = x\sigma(\beta x)$



常见激活函数

高斯误差线性单元 (Gaussian Error Linear Unit, GELU)

$$GELU(x) = xP(X \leq x) = x\Phi(x) = x \cdot \frac{1}{2} [1 + \operatorname{erf}(x/\sqrt{2})]$$

- 其中 $P(X \leq x)$ 是高斯分布 $N(\mu, \sigma^2)$ 的累积分布函数, 其中 μ, σ 为超参数, 一般设 $\mu = 0, \sigma = 1$ 即可
- 由于高斯分布的累积分布函数为S型函数, 因此GELU可以用Tanh函数或Logistic函数来近似

$$GELU(x) \approx 0.5x \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} (x + 0.044715x^3) \right) \right)$$

或 $GELU(x) \approx x\sigma(1.702x).$

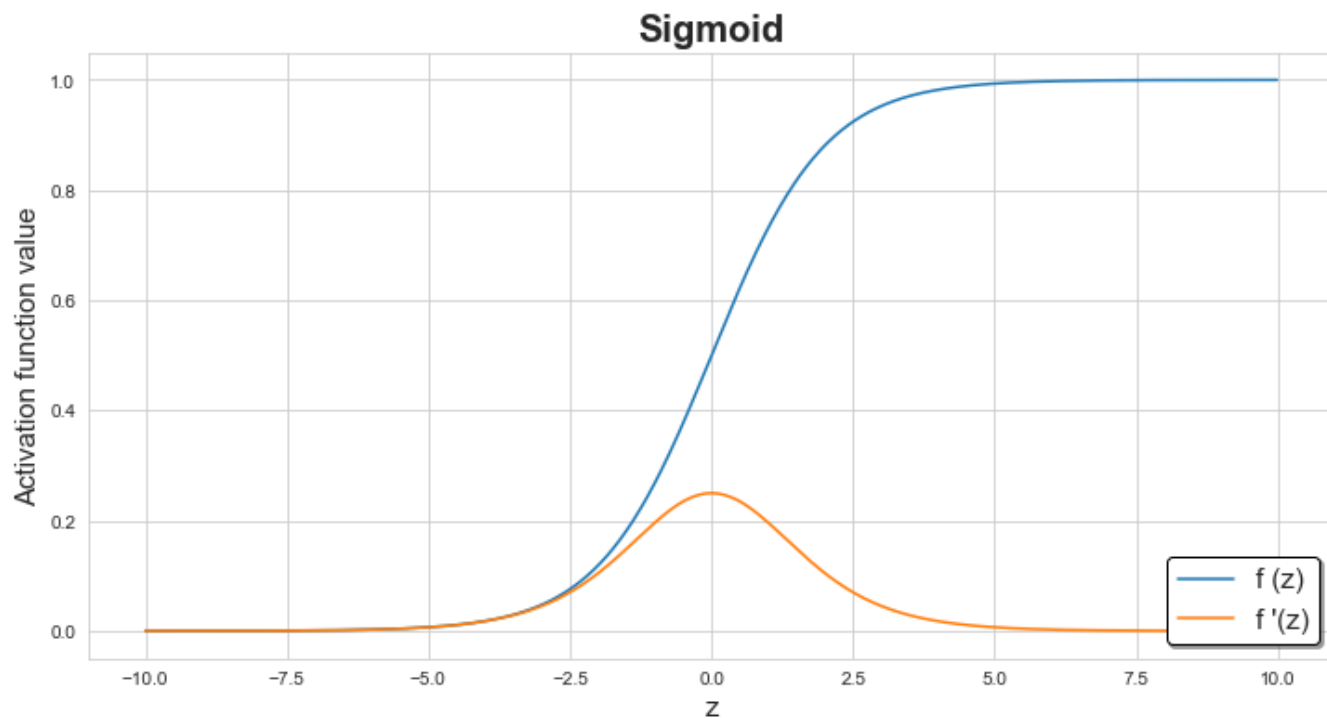
常见激活函数的导数

激活函数	函数	导数
Logistic 函数	$f(x) = \frac{1}{1+\exp(-x)}$	$f'(x) = f(x)(1 - f(x))$
Tanh 函数	$f(x) = \frac{\exp(x)-\exp(-x)}{\exp(x)+\exp(-x)}$	$f'(x) = 1 - f(x)^2$
ReLU 函数	$f(x) = \max(0, x)$	$f'(x) = I(x > 0)$
ELU 函数	$f(x) = \max(0, x) + \min(0, \gamma(\exp(x) - 1))$	$f'(x) = I(x > 0) + I(x \leq 0) \cdot \gamma \exp(x)$
SoftPlus 函数	$f(x) = \log(1 + \exp(x))$	$f'(x) = \frac{1}{1+\exp(-x)}$

sigmoid

$$f(x) = \frac{1}{1 + e^{-x}} \quad f'(x) = f(x) \cdot (1 - f(x))$$

```
1 z = np.arange(-10, 10, 0.01)
2 f = 1/(1 + np.exp(-z))
3 df = f*(1 - f)
4 make_plot(z, f, df, "Sigmoid")
```

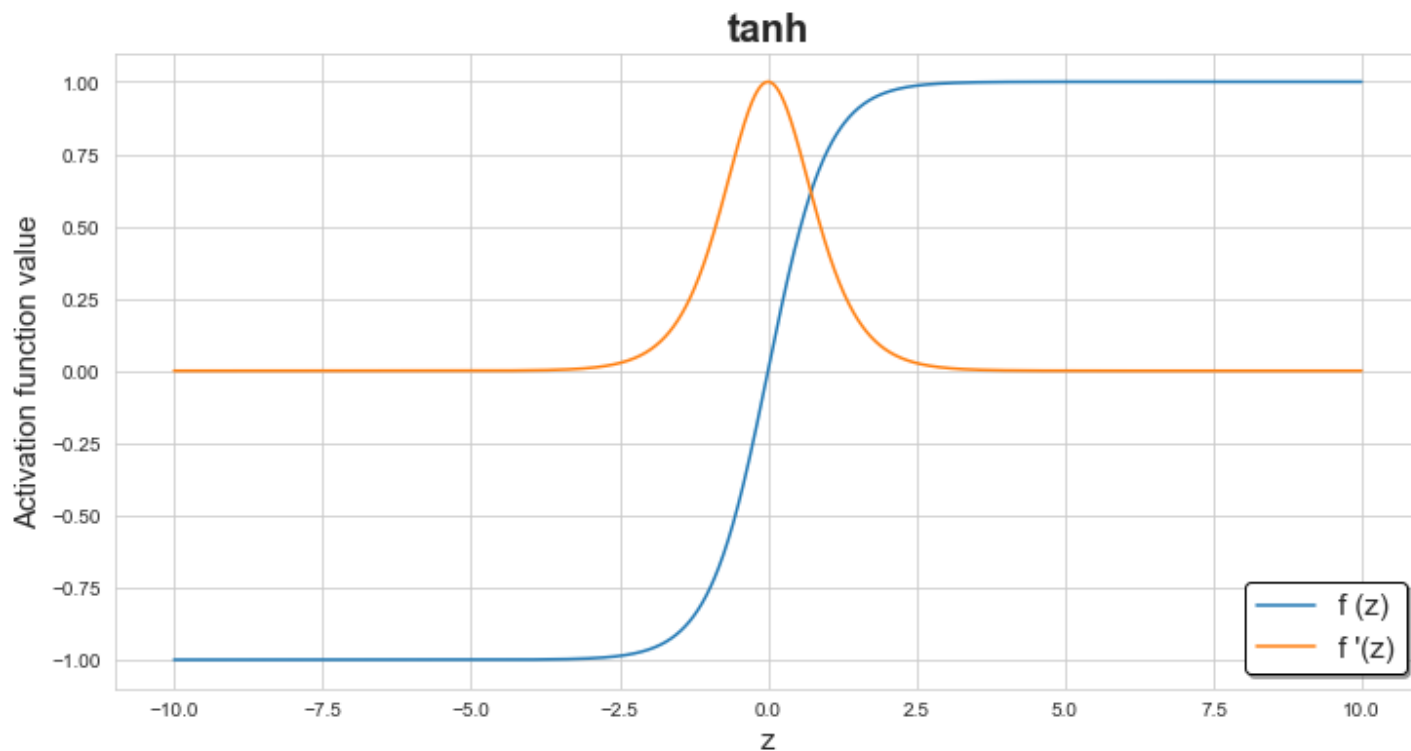


Tanh

$$f(x) = \tanh x = \frac{\sinh x}{\cosh x} = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

$$f'(x) = 1 - f(x)^2$$

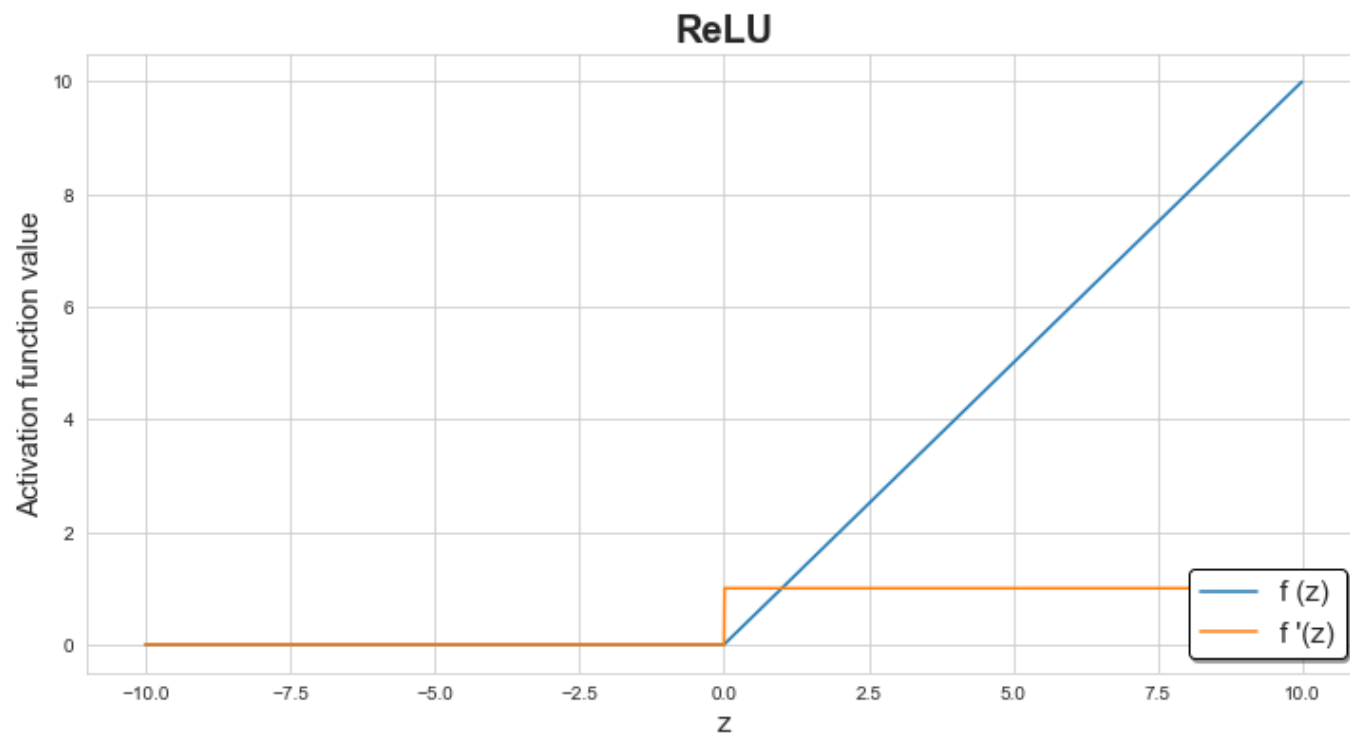
```
1 z = np.arange(-10, 10, 0.01)
2 f = np.tanh(z)
3 df = 1 - f*f
4 make_plot(z, f, df, "tanh")
```



ReLU

$$f(x) = \begin{cases} 0 & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases} \quad f'(x) = \begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x \geq 0 \end{cases}$$

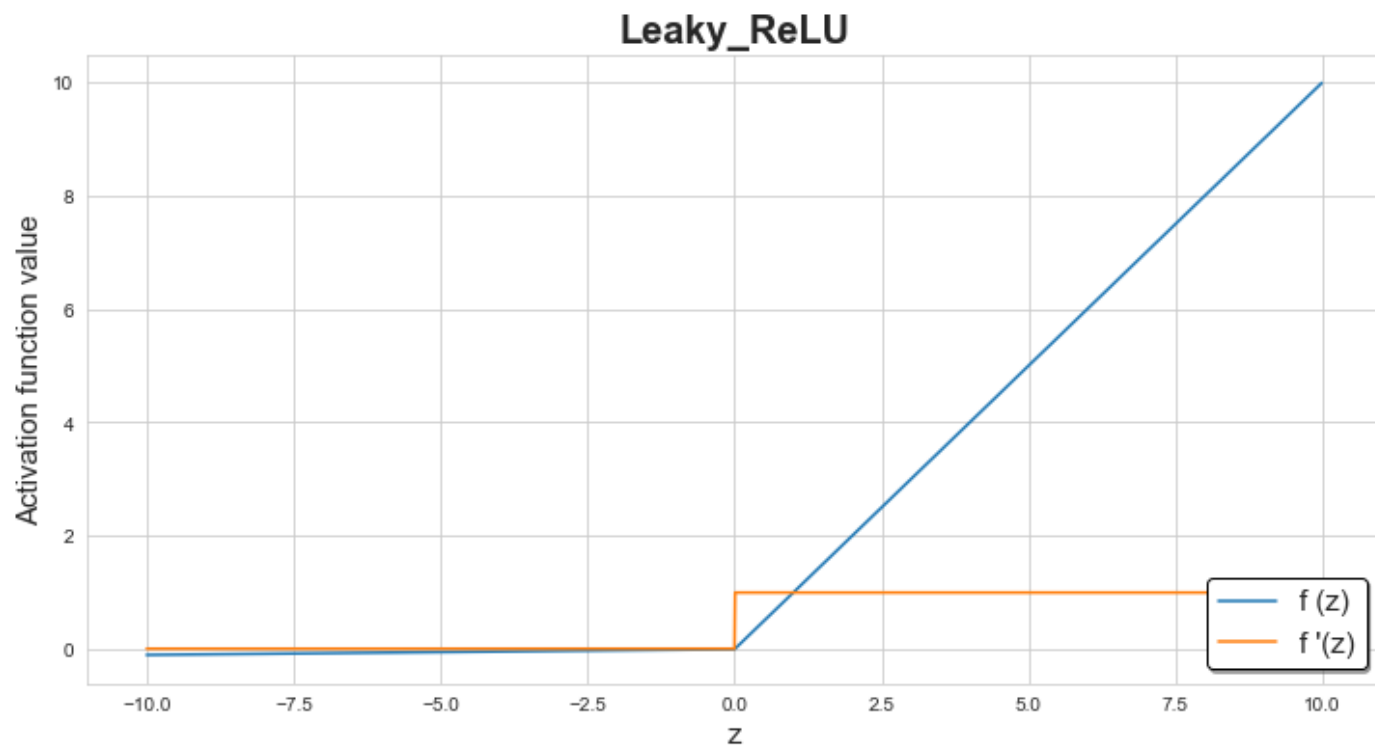
```
1 z = np.arange(-10, 10, 0.01)
2 f = z * (z > 0)
3 df = 1. * (z > 0)
4 make_plot(z, f, df, "ReLU")
```



Leaky ReLU

$$f(x) = \begin{cases} 0.01 \cdot x & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases} \quad f'(x) = \begin{cases} 0.01 & \text{if } x < 0 \\ 1 & \text{if } x \geq 0 \end{cases}$$

```
1 z = np.arange(-10, 10, 0.01)
2 f = np.where(z > 0, z, z * 0.01)
3 df = np.where(z > 0, 1, 0.01)
4 make_plot(z, f, df, "Leaky_ReLU")
```



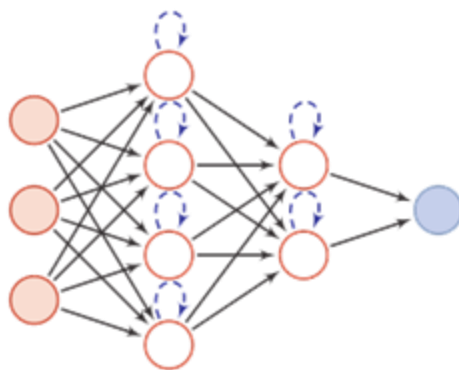
人工神经网络

人工神经网络主要由**大量的神经元**以及它们之间的**有向连接**构成。因此考虑三方面：

- 神经元的激活规则
 - 主要是指神经元输入到输出之间的映射关系（**激活函数**），一般为非线性函数。
- 网络的拓扑结构
 - 神经元之间如何连接（**DNN, CNN, RNN, GNN**）。
- 学习算法
 - 通过训练数据来学习神经网络的参数（**反向传播和梯度下降**）。



(a) 前馈网络

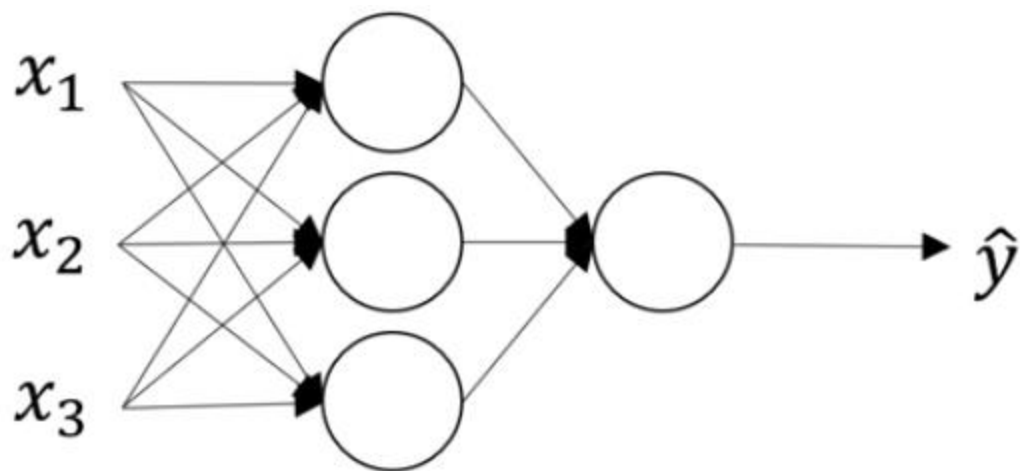


(b) 记忆网络



(c) 图网络

为什么需要激活函数



Given x :

$$z^{[1]} = W^{[1]}x + b^{[1]}$$

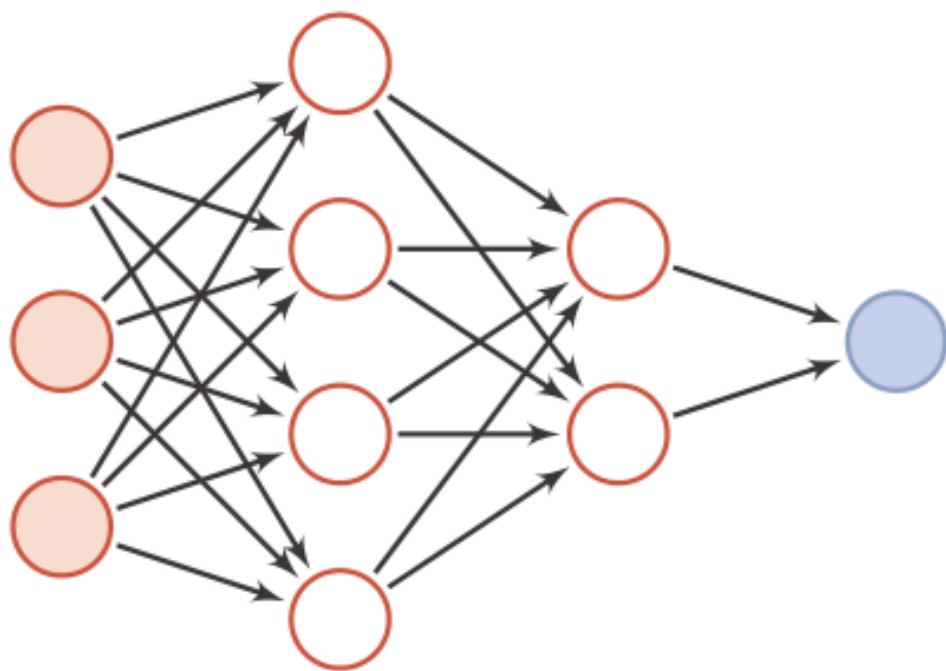
$$a^{[1]} = g^{[1]}(z^{[1]})$$

$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$$

$$a^{[2]} = g^{[2]}(z^{[2]})$$

神经网络本质

集成学习？特征提取？

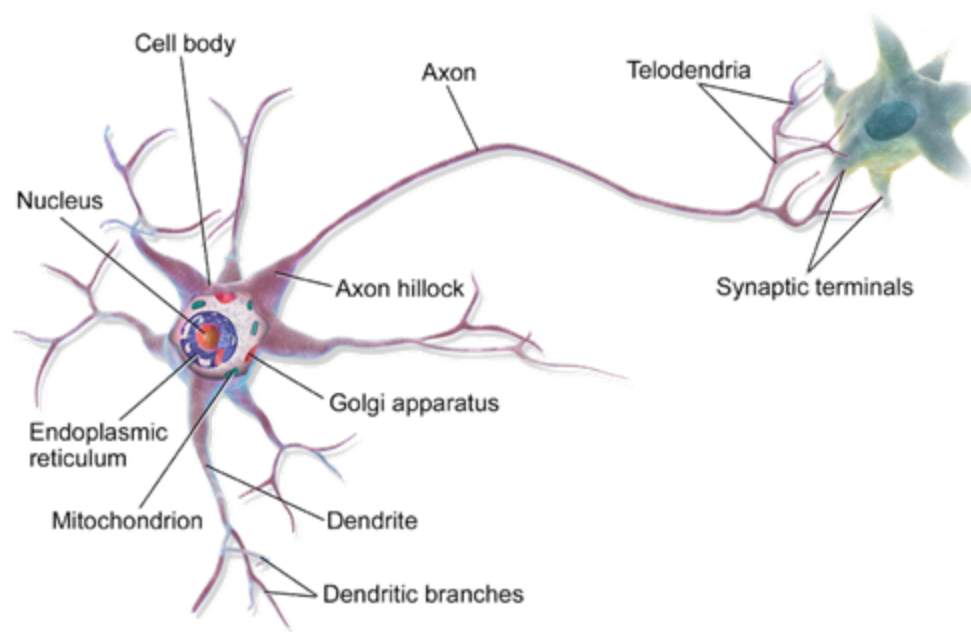
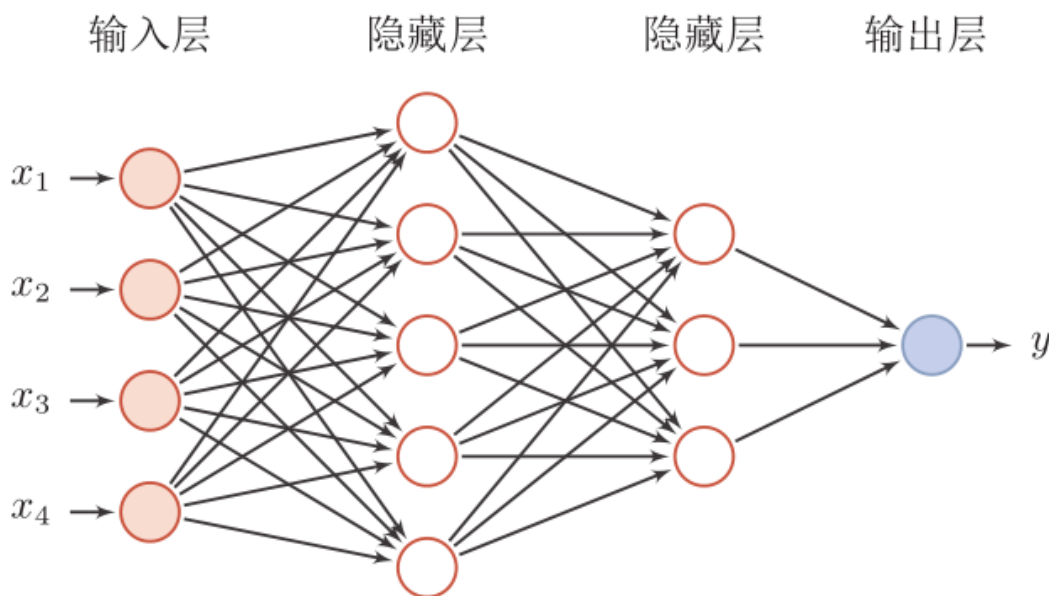


1
2
chapter

前馈神经网络

前馈神经网络（全连接神经网络、多层感知器）

- 各神经元分别属于不同的层，层内无连接。
- 相邻两层之间的神经元全部两两连接。
- 整个网络中无反馈，信号从输入层向输出层单向传播，可用一个有向无环图表示。

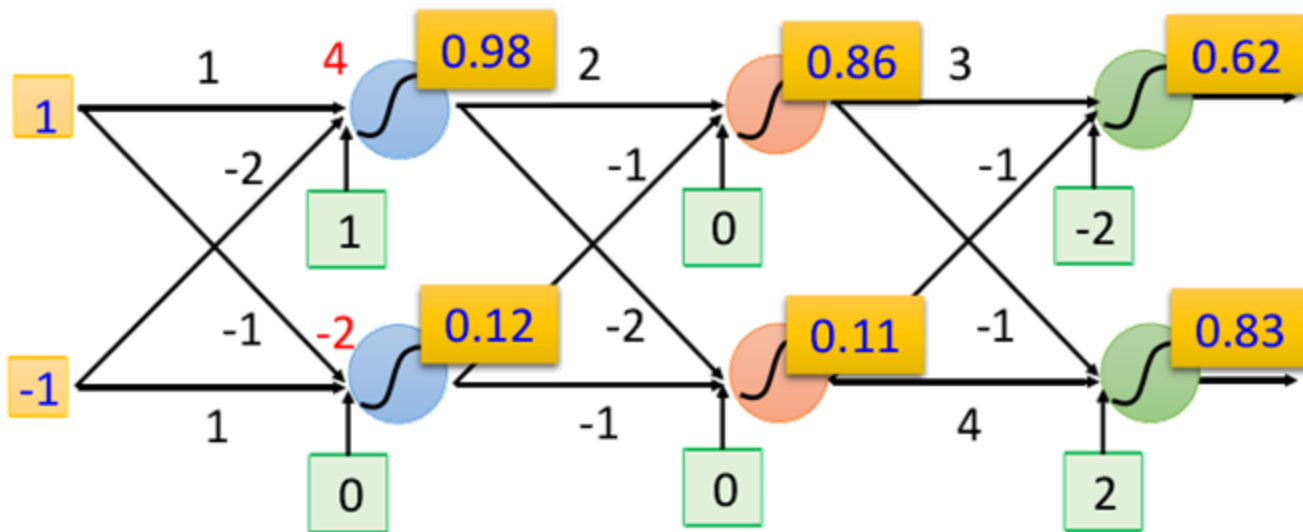
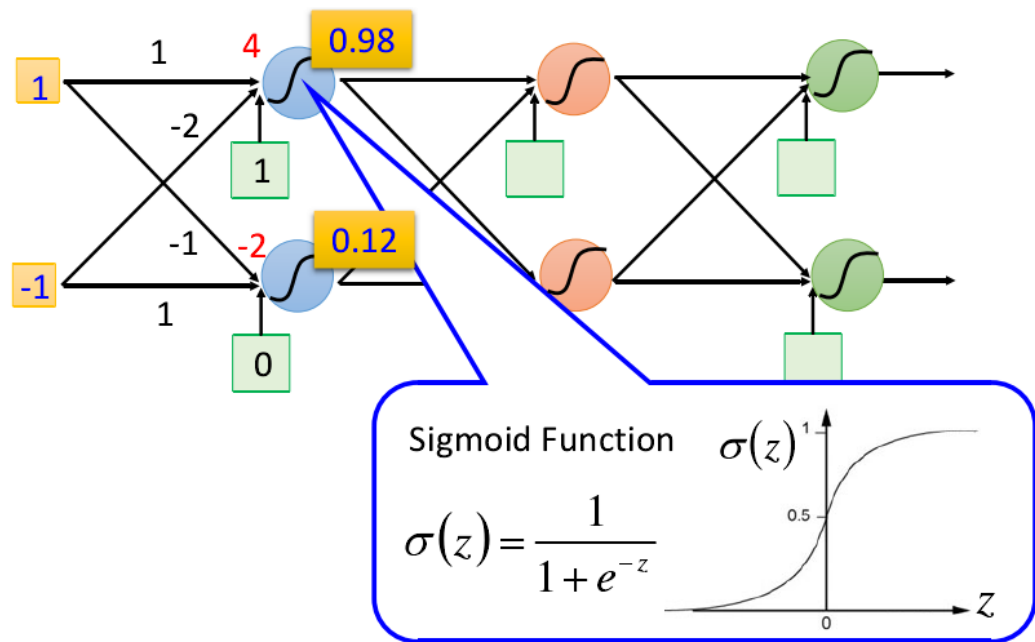


前馈计算

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)},$$

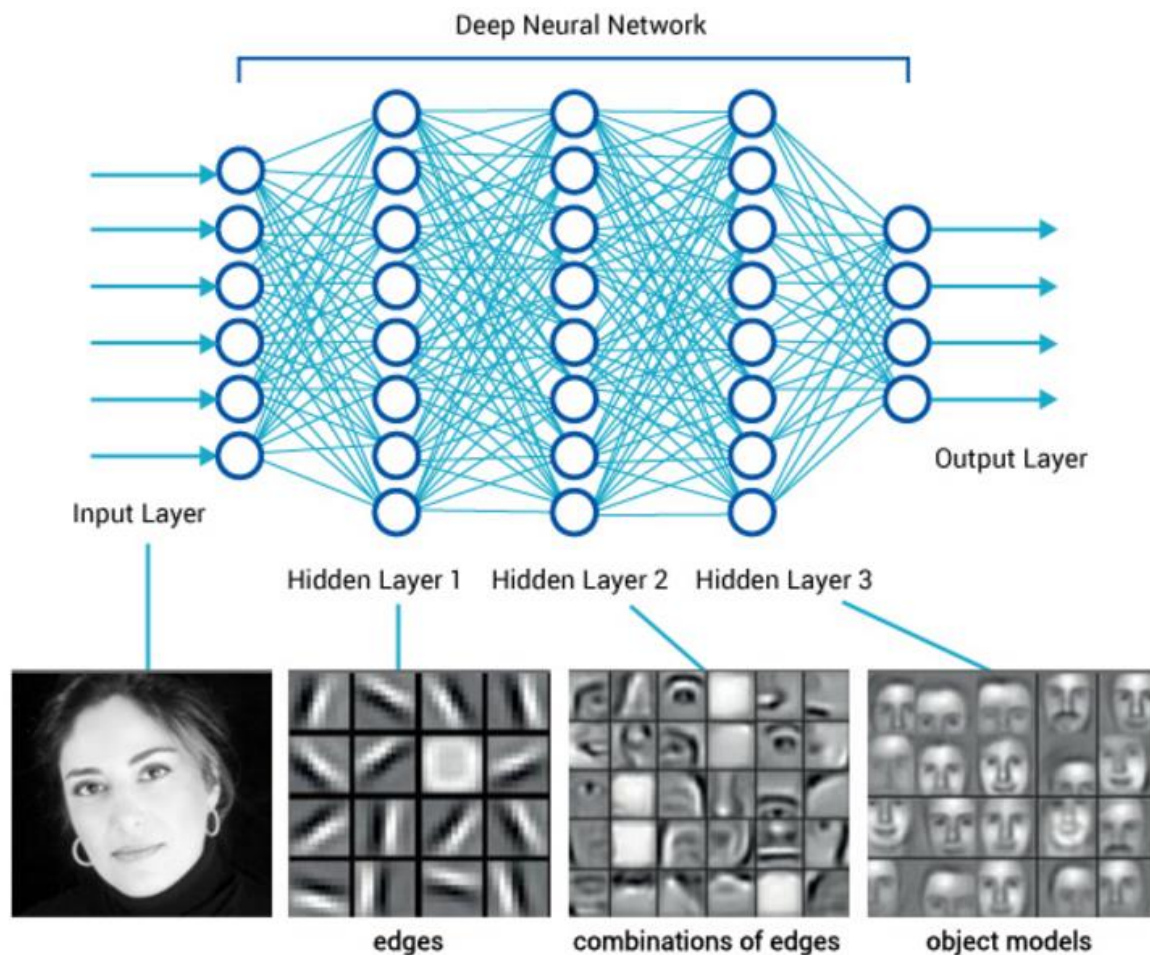
$$\mathbf{a}^{(l)} = f_l(\mathbf{z}^{(l)}).$$

$$\mathbf{x} = \mathbf{a}^{(0)} \rightarrow \mathbf{z}^{(1)} \rightarrow \mathbf{a}^{(1)} \rightarrow \mathbf{z}^{(2)} \rightarrow \dots \rightarrow \mathbf{a}^{(L-1)} \rightarrow \mathbf{z}^{(L)} \rightarrow \mathbf{a}^{(L)} = \phi(\mathbf{x}; \mathbf{W}, \mathbf{b})$$



深度神经网络

- 添加少量隐层的神经网络就叫做浅层神经网络；**隐层比较多的神经网络称为深度神经网络，俗称深度学习**



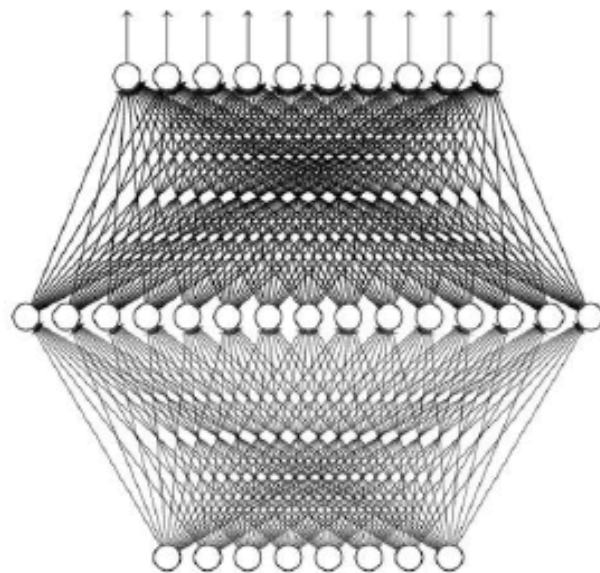
万能函数

Any continuous function f

$$f : R^N \rightarrow R^M$$

Can be realized by a network
with one hidden layer

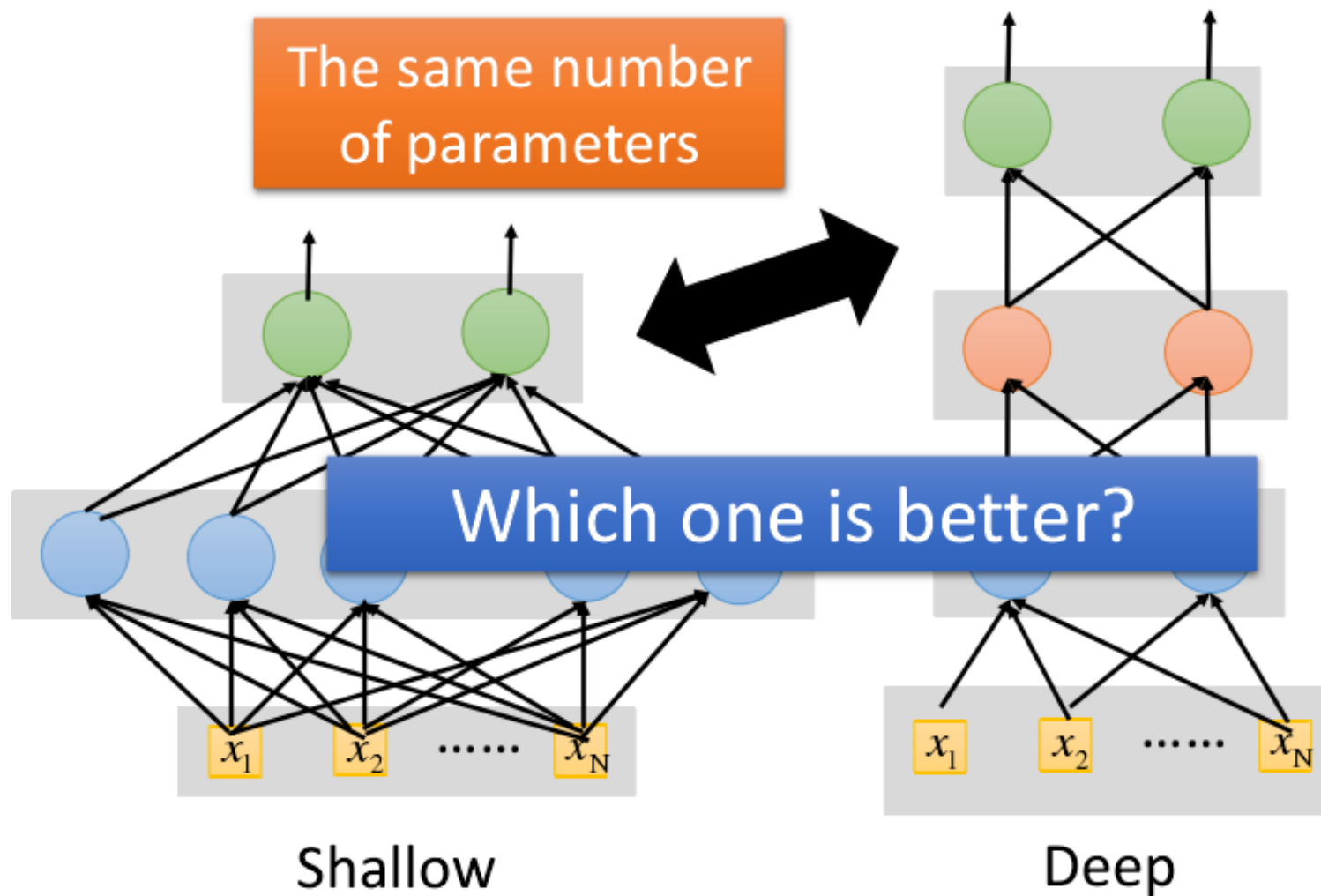
(given **enough** hidden
neurons)



Reference for the reason:
<http://neuralnetworksanddeeplearning.com/chap4.html>

Why “Deep” neural network not “Fat” neural network?

“深” 还是 “宽”



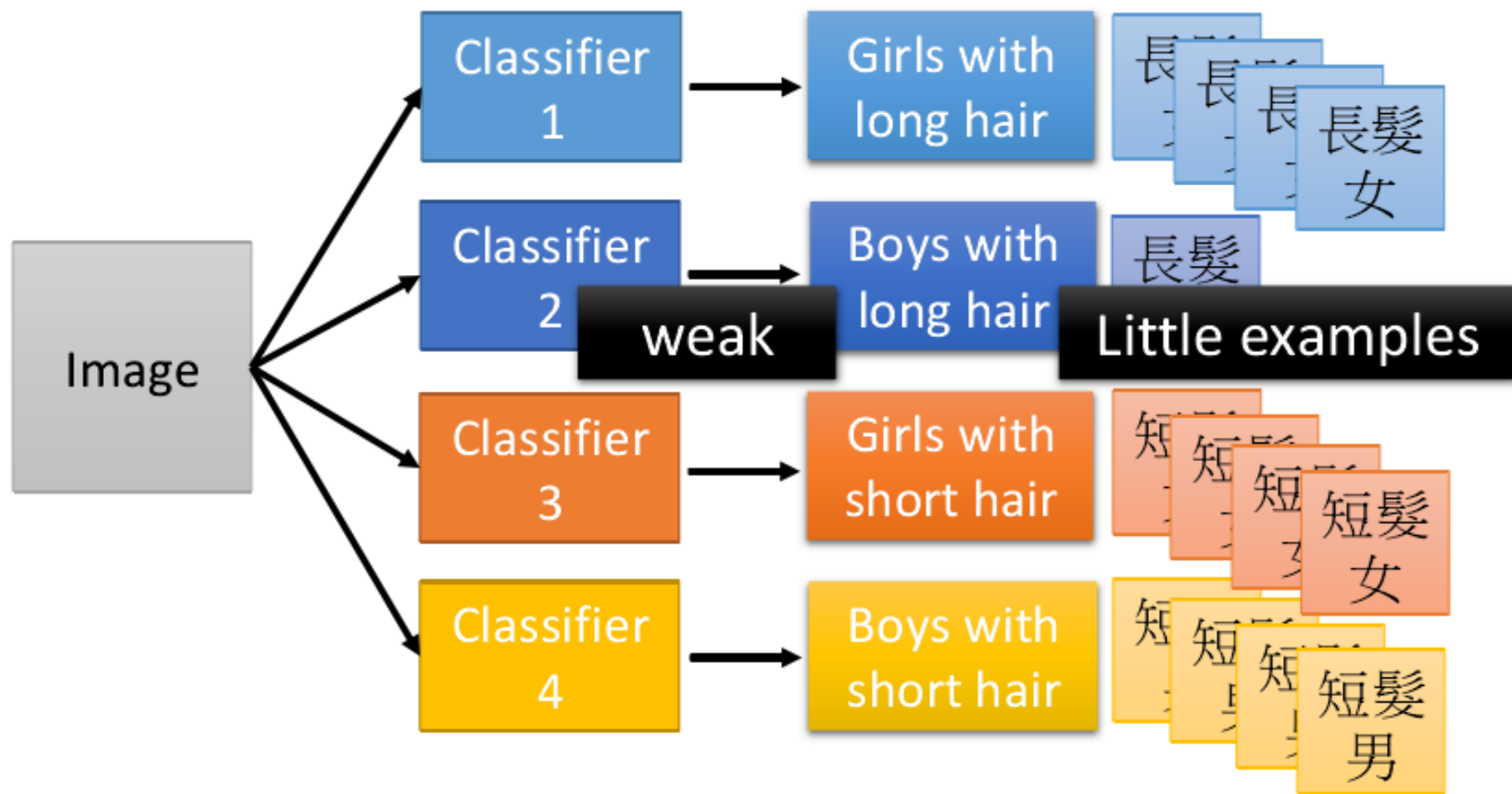
“深” 还是 “宽”

Layer X Size	Word Error Rate (%)	Layer X Size	Word Error Rate (%)
1 X 2k	24.2		
2 X 2k	20.4		
3 X 2k	18.4		
4 X 2k	17.8		
5 X 2k	17.2	1 X 3772	22.5
7 X 2k	17.1	1 X 4634	22.6
		1 X 16k	22.1

Seide, Frank, Gang Li, and Dong Yu. "Conversational Speech Transcription Using Context-Dependent Deep Neural Networks." *Interspeech*. 2011.

“深” 还是 “宽”

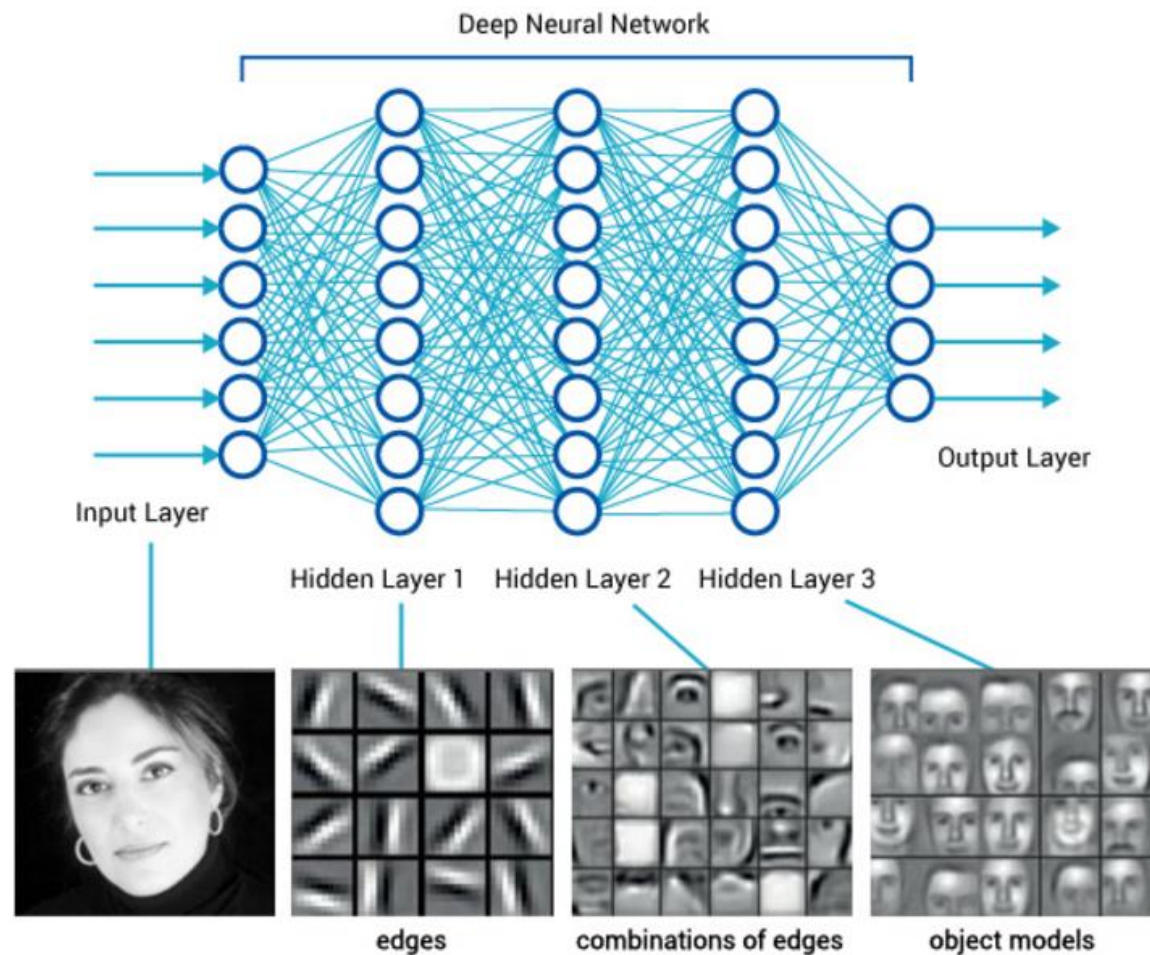
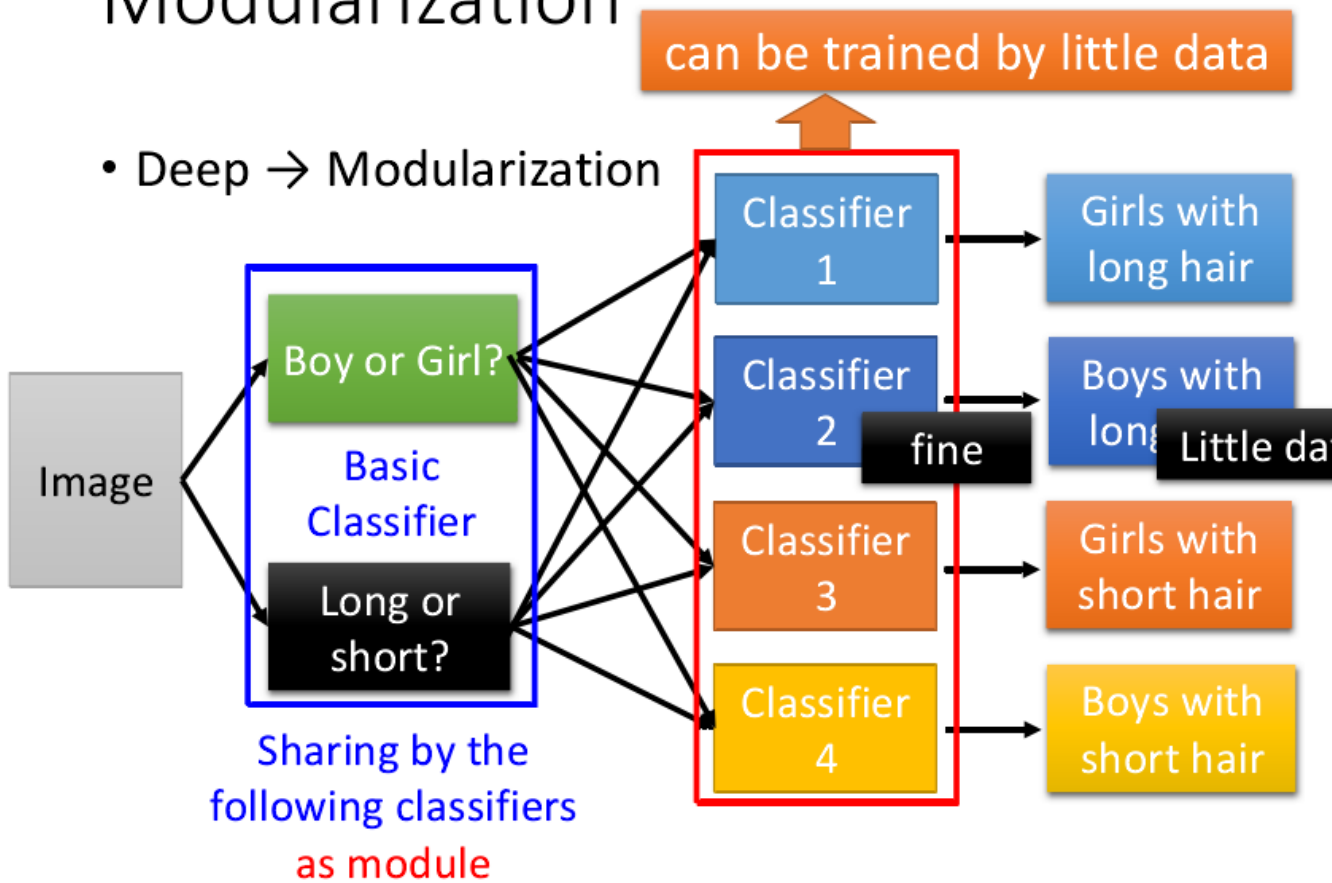
- Deep → Modularization



“深” 还是 “宽”

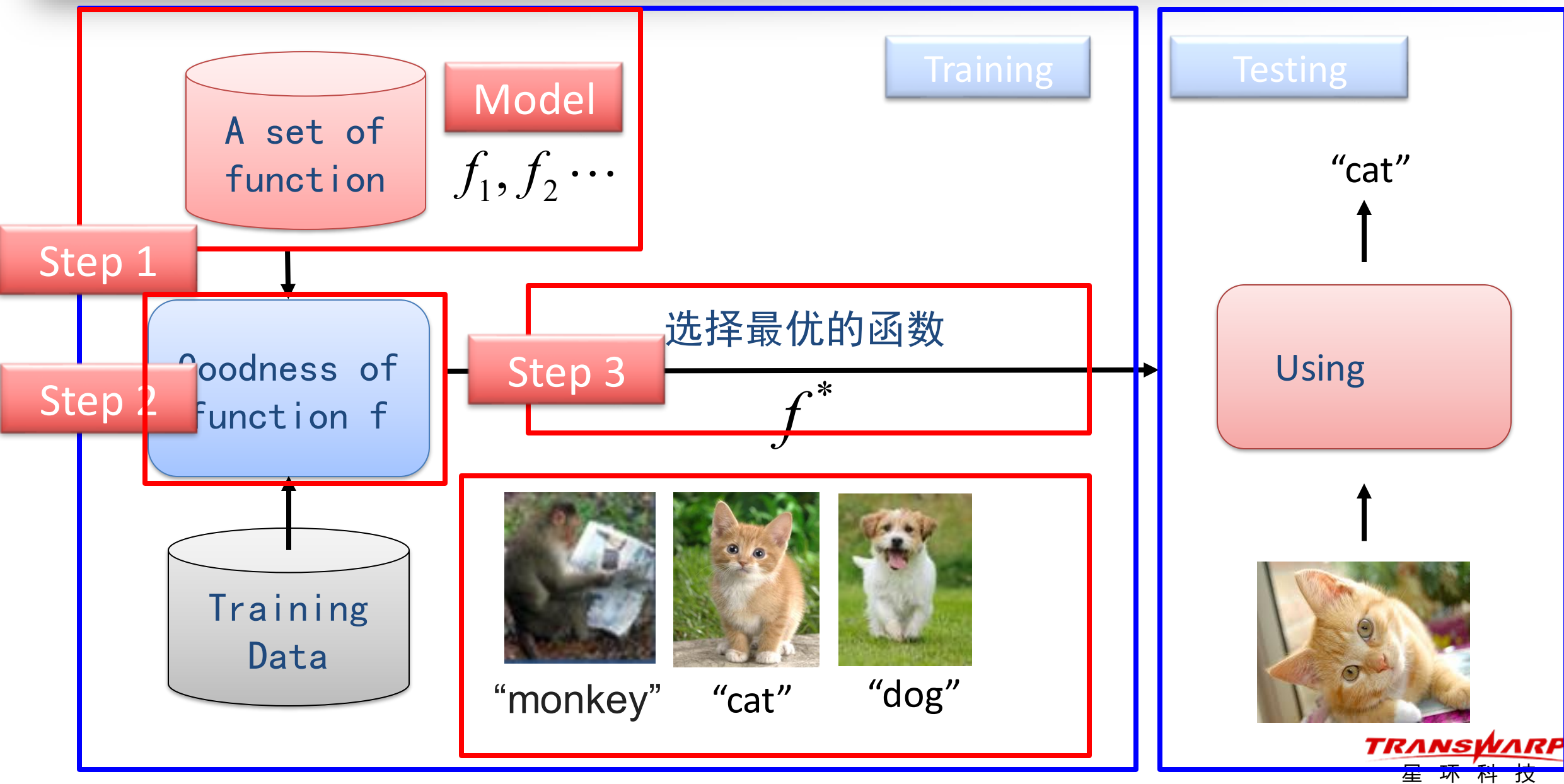
Modularization

- Deep → Modularization



1
3
chapter

参数学习



神经网络与逻辑回归

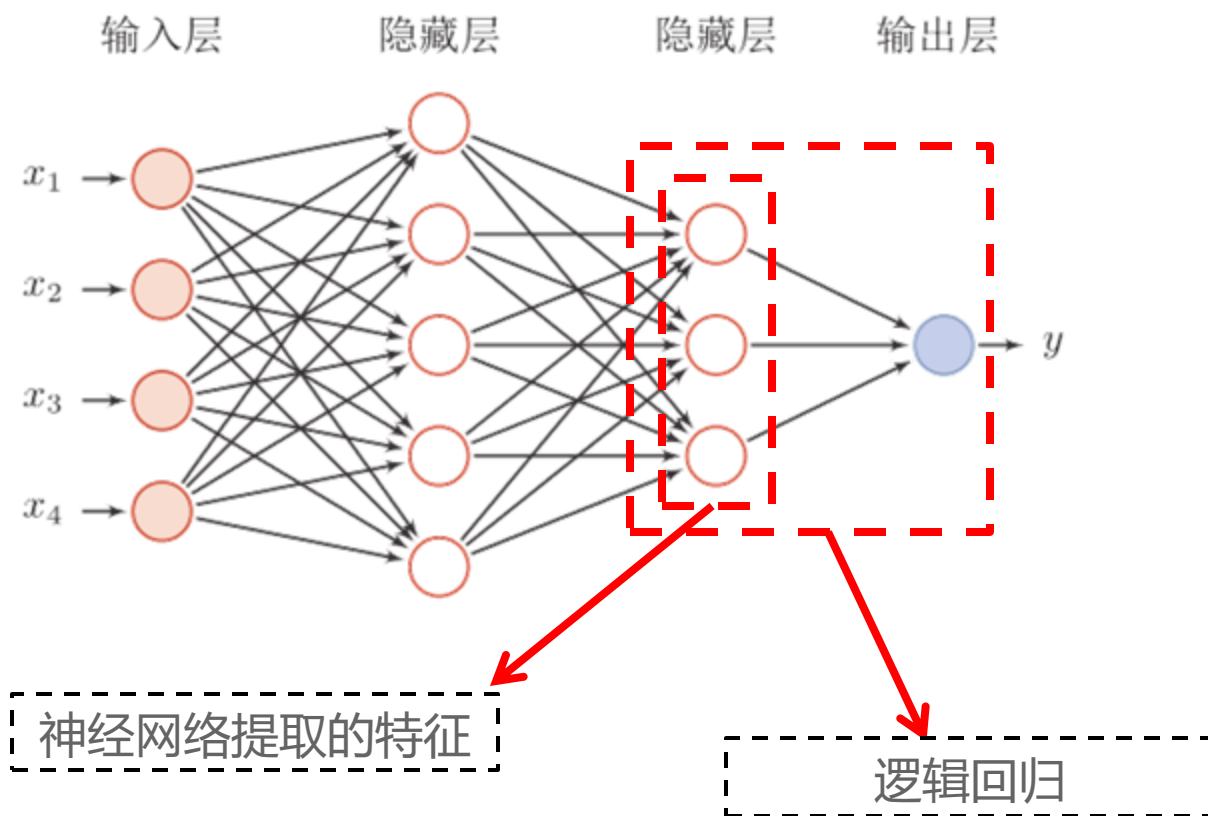
- 神经网络可以作为一个“万能”函数来使用，可以用来进行复杂的特征转换，或逼近一个复杂的条件分布。

$$\hat{y} = g(\underline{\varphi(\mathbf{x})}, \theta)$$

分类器

神经网络

如果 $g(\cdot)$ 为Logistic回归，那么神经网络的最后一层可以看成是Logistic回归分类器。



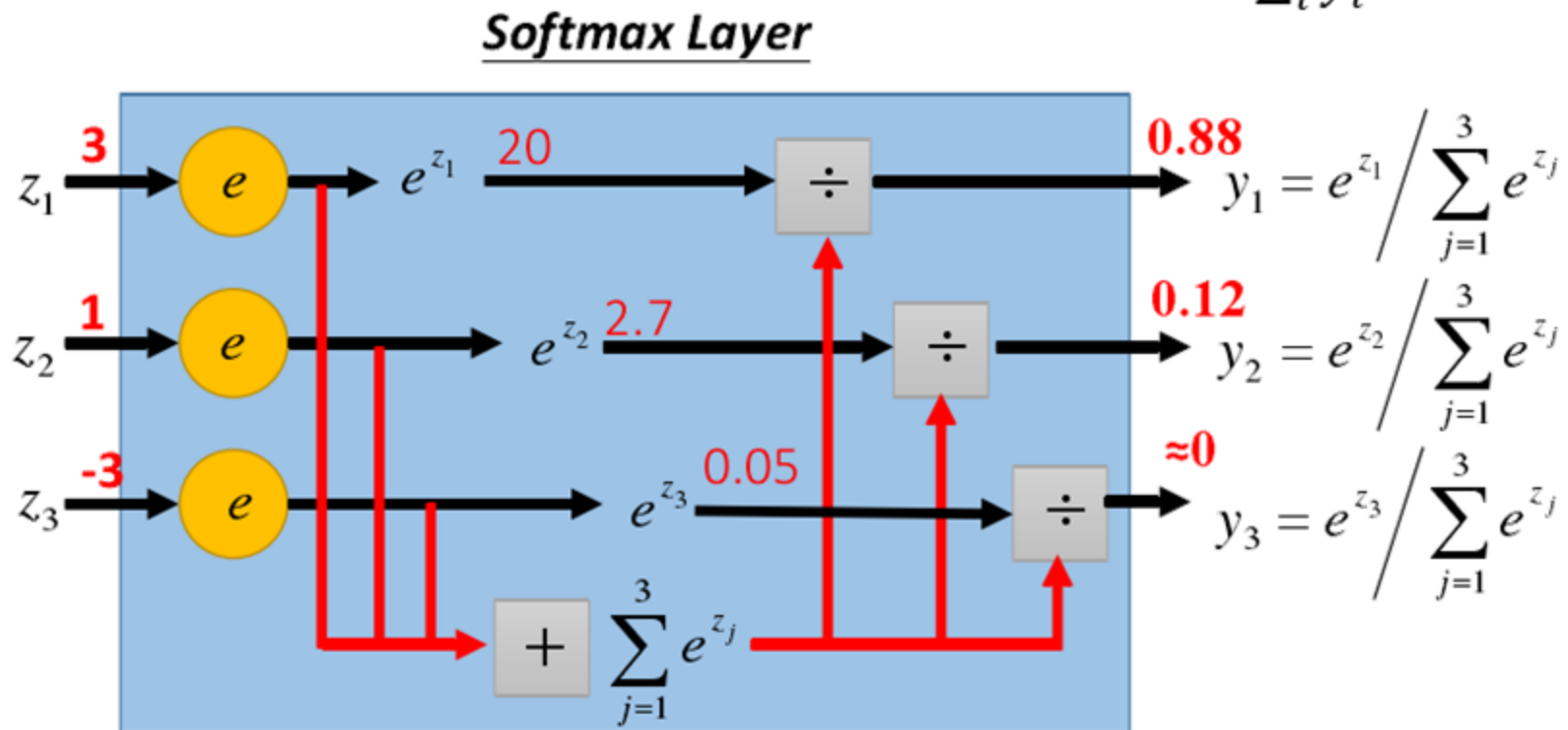
Softmax概率计算

- softmax函数计算输出概率

- Softmax layer as the output layer

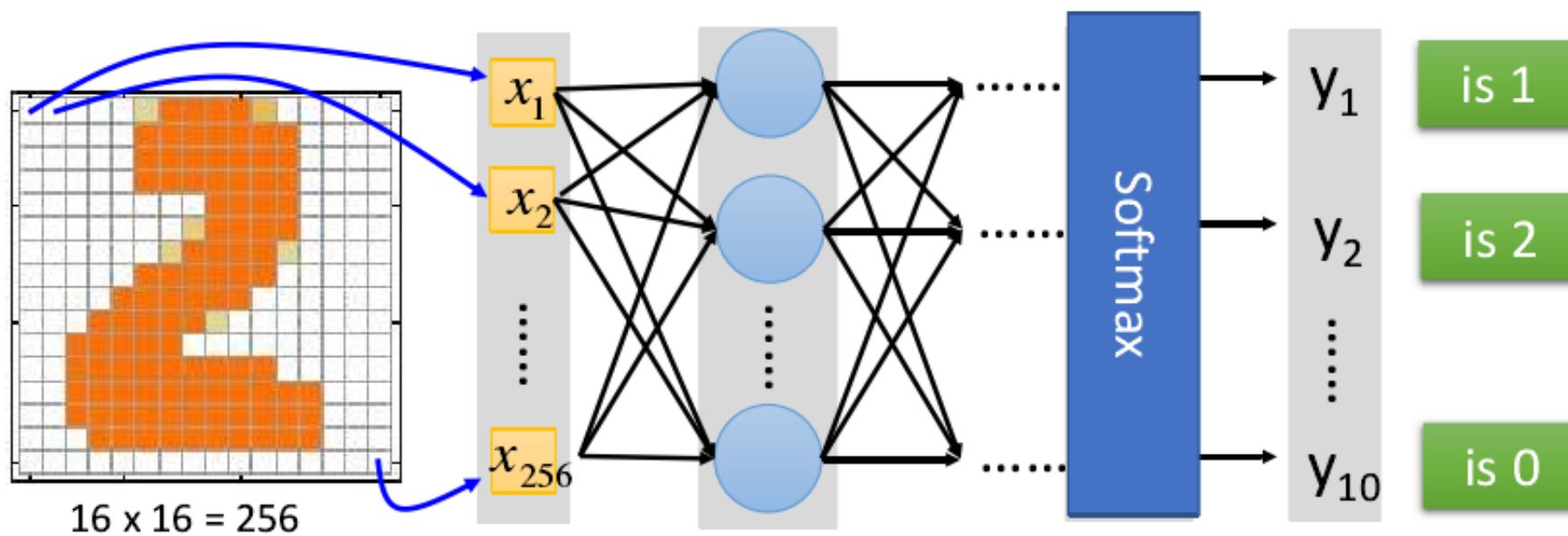
Probability:

- $1 > y_i > 0$
- $\sum_i y_i = 1$



学习目标

- 更准确的预测真实值



16 x 16 = 256

Ink $\rightarrow$ 1

No ink $\rightarrow$ 0

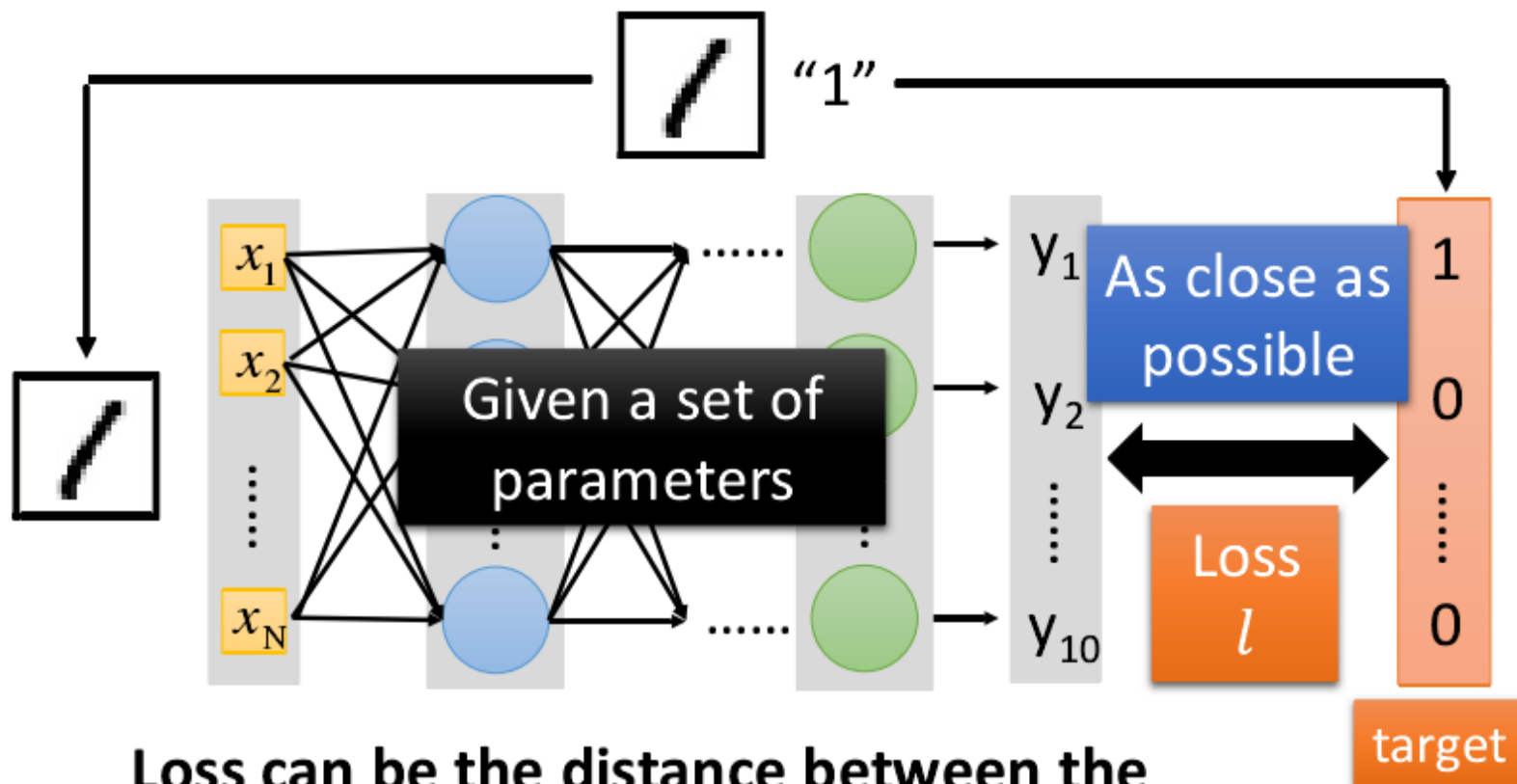
The learning target is

Input:  $\rightarrow$ y_1 has the maximum value

Input:  $\rightarrow$ y_2 has the maximum value

单个样本损失计算

- **单个样本损失**=预测值和真实值之间计算误差
- 好的模型函数能够让损失尽量小



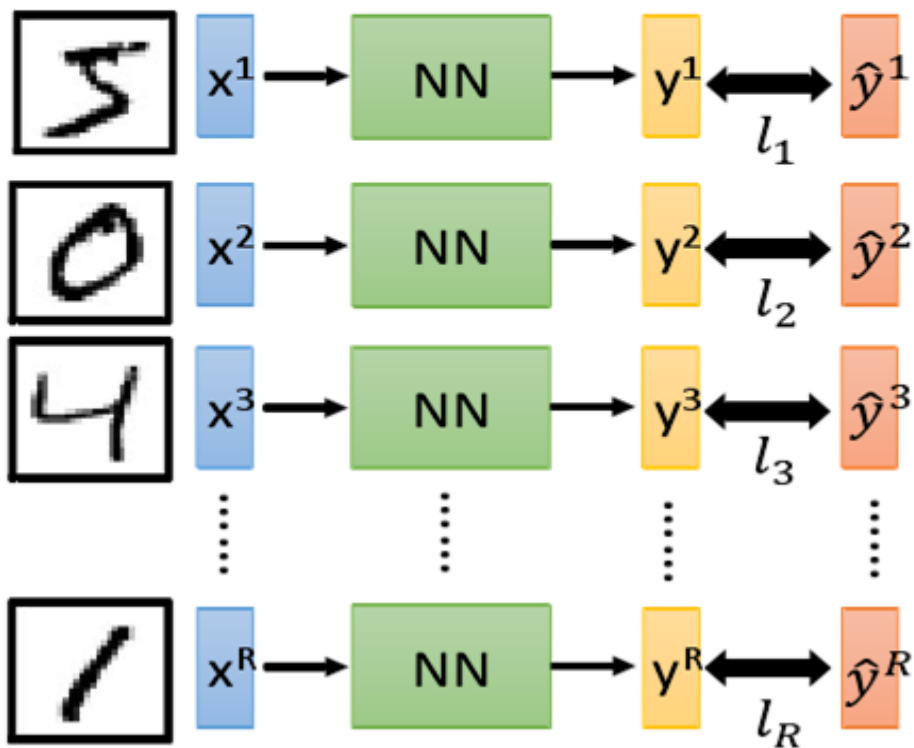
Loss can be the distance between the network output and target

总损失计算

- 总损失=所有样本损失之和 (或平均值)

Total Loss

For all training data ...



Total Loss:

$$L = \sum_{r=1}^R l_r$$

As small as possible

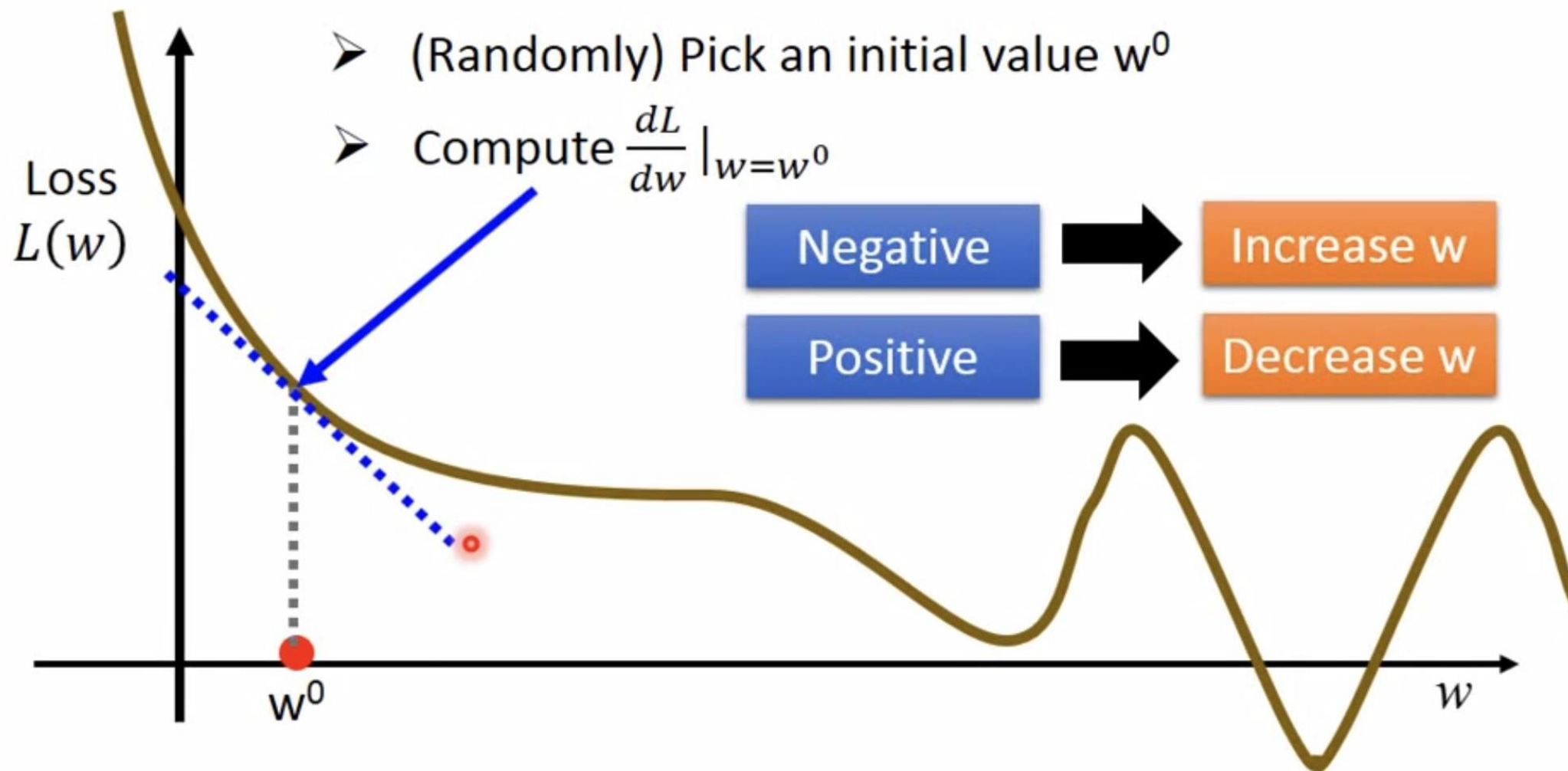
Find a function in function set that minimizes total loss L Find the network parameters θ^* that minimize total loss L

使用梯度下降法求解参数

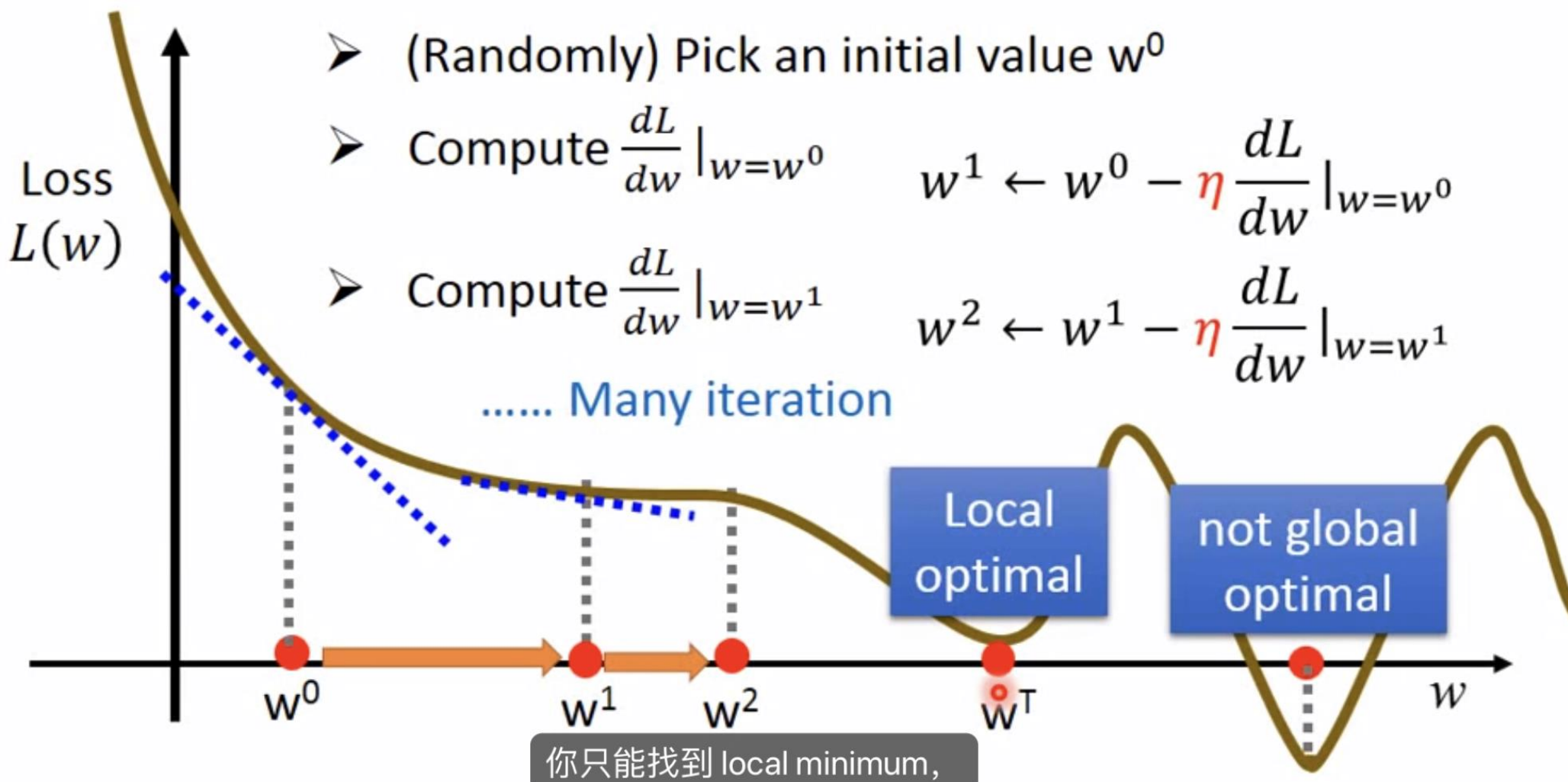
- 如何求导数： **反向传播**算法
- 如何更新参数： 各种**优化器**

梯度下降

- Consider loss function $L(w)$ with one parameter w :



梯度下降



梯度下降的三种形式

- **批量梯度下降** (Batch Gradient Descent, BGD) 梯度下降的每一步中, 都用到了所有的训练样本
- **随机梯度下降** (Stochastic Gradient Descent, SGD) 梯度下降的每一步中, 用到一个样本, 在每一次计算之后便更新参数, 而不需要首先将所有的训练集求和
- **小批量梯度下降** (Mini-Batch Gradient Descent, MBGD) 梯度下降的每一步中, 用到了一定批量的训练样本

批量梯度下降 (Batch Gradient Descent, BGD)

- 梯度下降的每一步中，都用到了**所有的**训练样本

参数更新

$$w_j := w_j - \alpha \frac{1}{m} \sum_{i=1}^m \left((h(x^{(i)}) - y^{(i)}) \cdot x_j^{(i)} \right)$$

(同步更新 w_j , $(j=0,1,...,n)$)

学习率

梯度

随机梯度下降 (Stochastic Gradient Descent, SGD)

- 梯度下降的每一步中，用到一个样本，在每一个样本计算之后边更新参数，而不是需要首先将所有样本求和

参数更新

$$w_j := w_j - \alpha (h(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

(同步更新 w_j , $(j=0,1,...,n)$)

小批量梯度下降 (Mini-Batch Gradient Descent, MBGD)

- 梯度下降的每一步中，用到**一定批量的**训练样本
- 每计算常数**b**次训练示例，便更新一次参数w

参数更新

$$w_j := w_j - \alpha \frac{1}{b} \sum_{k=i}^{i+b-1} (h(x^{(k)}) - y^{(k)}) x_j^{(k)}$$

(同步更新 w_j , $(j=0,1,...,n)$)

$b=1$ (随机梯度下降, SGD)

$b=m$ (批量梯度下降, BGD)

$b=\text{batch_size}$, 通常是2的指数倍, 常见有32, 64, 128等。 (小批量梯度下降, MBGD)

如何计算梯度？

- **链式法则**：神经网络是一个复杂的**复合函数**

$$y = f^5(f^4(f^3(f^2(f^1(x)))))) \rightarrow \frac{\partial y}{\partial x} = \frac{\partial f^1}{\partial x} \frac{\partial f^2}{\partial f^1} \frac{\partial f^3}{\partial f^2} \frac{\partial f^4}{\partial f^3} \frac{\partial f^5}{\partial f^4}$$

- **反向传播算法**：根据前馈网络的特点而设计用来**计算梯度**的**高效方法**
- **计算图**和**自动微分**

矩阵微积分

- 矩阵微积分 (Matrix Calculus) 是多元微积分的一种表达方式, 即使用矩阵和向量来表示因变量**每个成分**关于自变量每个成分的偏导数。

- **标量关于向量的偏导数**

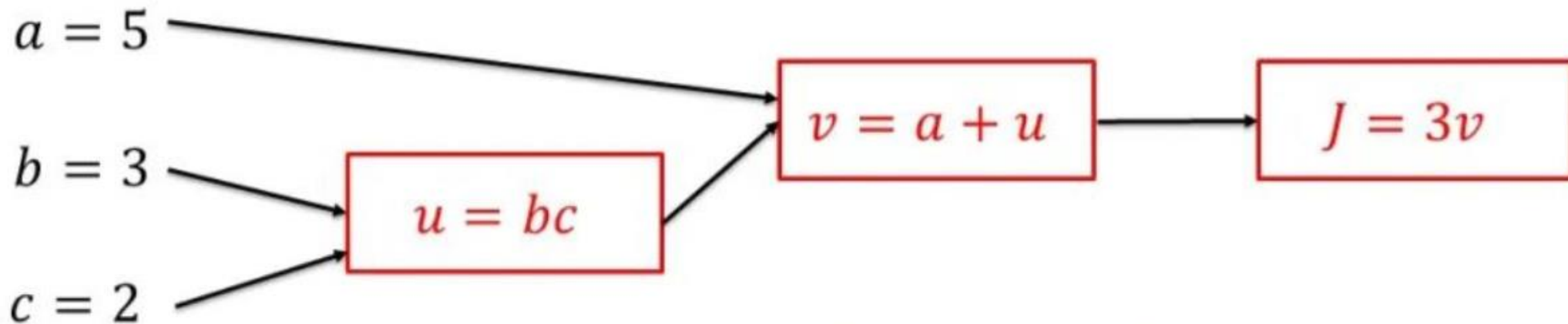
$$\frac{\partial y}{\partial \mathbf{x}} = \left[\frac{\partial y}{\partial x_1}, \dots, \frac{\partial y}{\partial x_p} \right]^T$$

- **向量关于向量的偏导数**

$$\frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \dots & \frac{\partial y_q}{\partial x_1} \\ \vdots & \vdots & \vdots \\ \frac{\partial y_1}{\partial x_p} & \dots & \frac{\partial y_q}{\partial x_p} \end{bmatrix} \in \mathbb{R}^{p \times q}$$

计算图

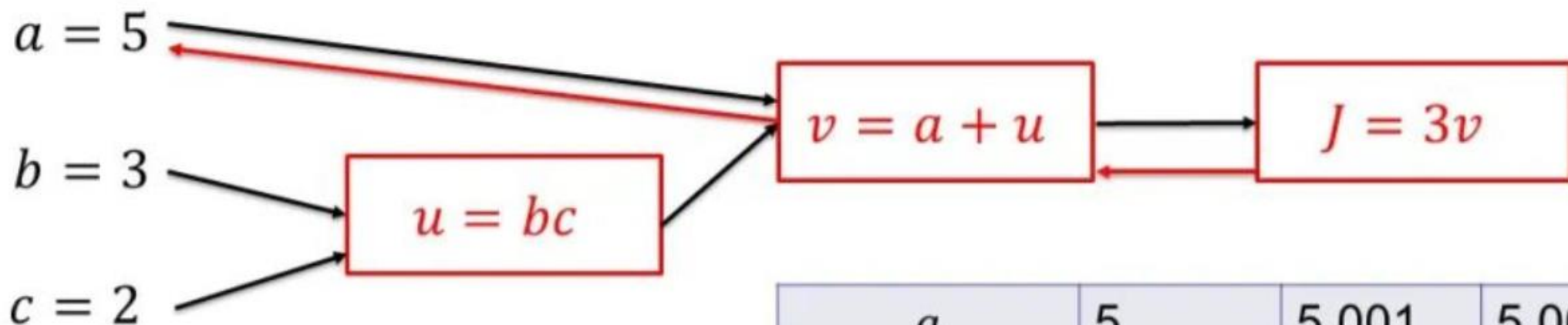
$$J(a, b, c) = 3(a + bc), a = 5, b = 3, c = 2$$



u	6
v	11
J	33

计算图

$$J(a, b, c) = 3(a + bc) \quad \frac{dJ}{du} = \frac{dJ}{dv} \frac{dv}{du}, \quad \frac{dJ}{db} = \frac{dJ}{du} \frac{du}{db}, \quad \frac{dJ}{da} = \frac{dJ}{du} \frac{du}{da}$$



$$\frac{dJ}{dv} = 3, \quad \frac{dv}{da} = 1$$

a	5	5.001	5.002
u	6	6.001	6.002
v	11	11.001	11.002
J	33	33.003	33.006

自动微分

前向模式和反向模式

- **反向模式**和**反向传播**的计算梯度的方式相同
- 如果函数和参数之间有**多条路径**，可以将这多条路径上的**导数再进行相加**，得到**最终的梯度**。

静态计算图和动态计算图

- 静态计算图是在编译时构建计算图，计算图构建好之后在程序运行时不能改变。
 - Theano和Tensorflow
- 动态计算图是在程序运行时动态构建。两种构建方式各有优缺点。
 - DyNet, Chainer和PyTorch
- 静态计算图在构建时可以进行优化，并行能力强，但灵活性比较差。动态计算图则不容易优化，当不同输入的网络结构不一致时，难以并行计算，但是灵活性比较高。

静态计算图和动态计算图

TensorFlow:先搭建所有的计算图之后, 再把数据输入进去

```
N, D, H = 64, 1000, 100
x = tf.placeholder(tf.float32, shape=(N, D))
y = tf.placeholder(tf.float32, shape=(N, D))
w1 = tf.Variable(tf.random_normal((D, H)))
w2 = tf.Variable(tf.random_normal((H, D)))

h = tf.maximum(tf.matmul(x, w1), 0)
y_pred = tf.matmul(h, w2)
diff = y_pred - y
loss = tf.reduce_mean(tf.reduce_sum(diff ** 2, axis=1))
grad_w1, grad_w2 = tf.gradients(loss, [w1, w2])

learning_rate = 1e-5
new_w1 = w1.assign(w1 - learning_rate * grad_w1)
new_w2 = w2.assign(w2 - learning_rate * grad_w2)
updates = tf.group(new_w1, new_w2)

with tf.Session() as sess:
    sess.run(tf.global_variables_initializer())
    values = {x: np.random.randn(N, D),
              y: np.random.randn(N, D),}
    losses = []
    for t in range(50):
        loss_val, _ = sess.run([loss, updates],
                               feed_dict=values)
```

创建图

运行每次迭代

PyTorch:动态图的搭建是根据每一步的计算搭建的

```
import torch

N, D_in, H, D_out = 64, 1000, 100, 10
x = torch.randn(N, D_in)
y = torch.randn(N, D_out)
w1 = torch.randn(D_in, H, requires_grad=True)
w2 = torch.randn(H, D_out, requires_grad=True)

learning_rate = 1e-6
for t in range(500):
    y_pred = x.mm(w1).clamp(min=0).mm(w2)
    loss = (y_pred - y).pow(2).sum()

    loss.backward()
```

每次迭代中创建图

神经网络训练流程

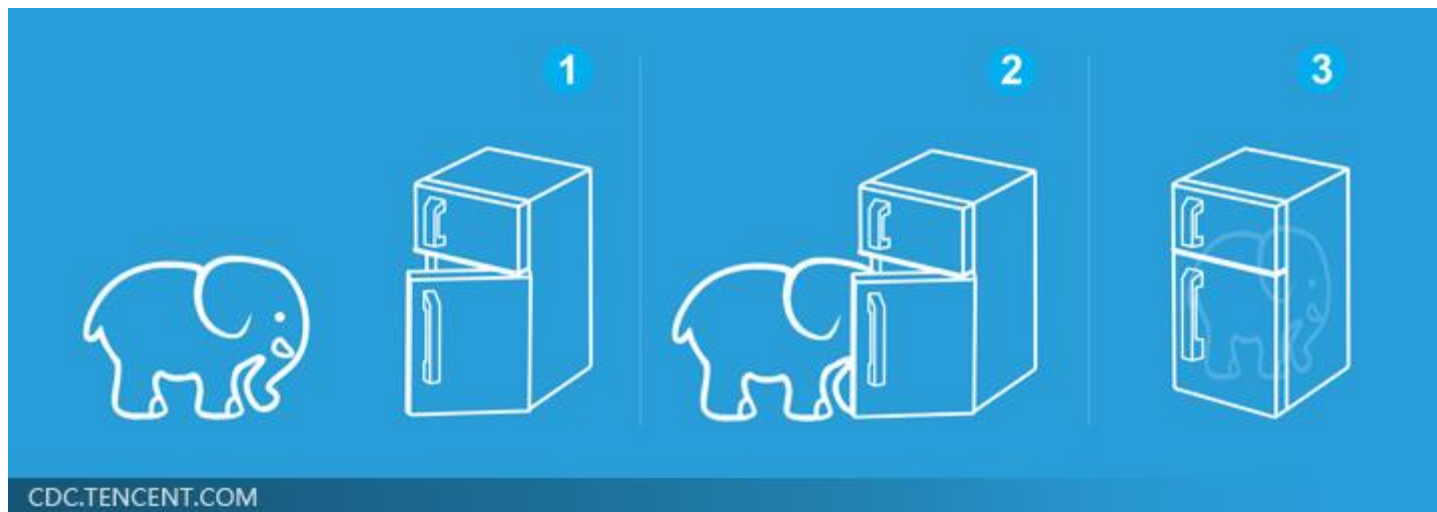
神经网络的训练过程可以分为以下三步

- **前向计算**每一层的状态和激活值，直到最后一层
- **反向计算**每一层的参数的偏导数
- **更新参数**

深度学习的三个步骤



Deep Learning is so simple

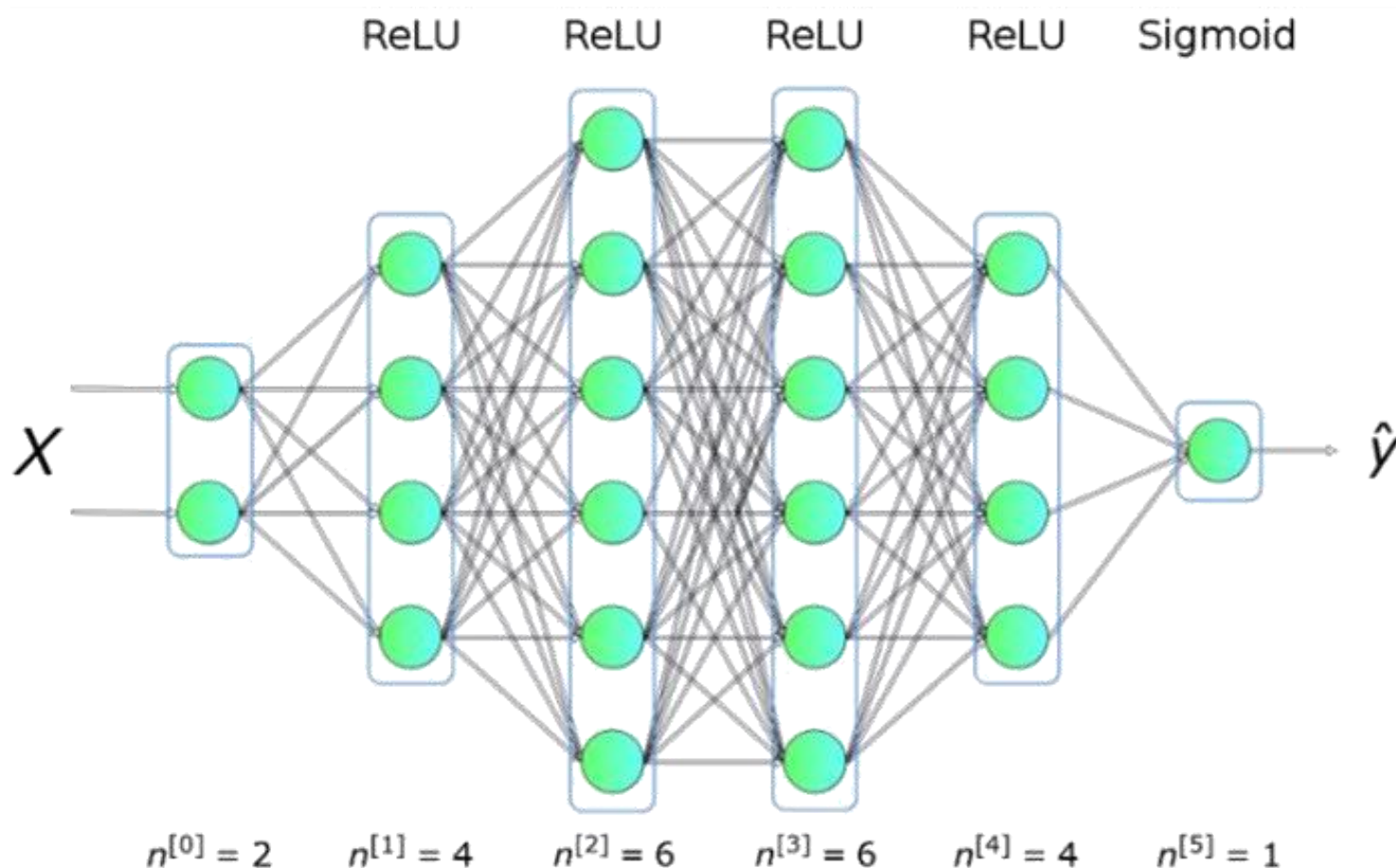


The background features a complex network diagram with numerous nodes (small circles) and connecting lines (edges) forming a web-like structure. The nodes are distributed across the entire frame, with a higher density of connections on the left and right sides, creating a sense of depth and connectivity.

4 chapter

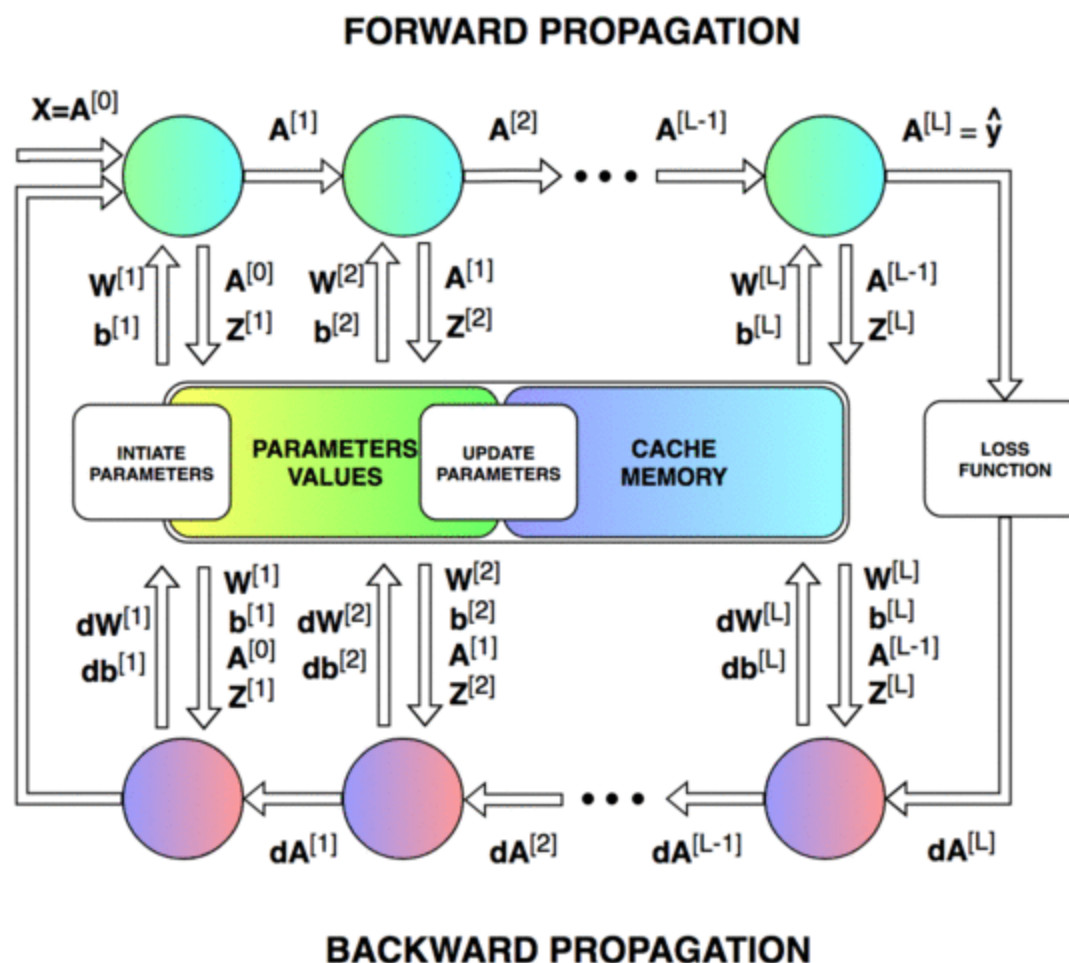
Numpy手动实现神经网络

Keras、TensorFlow、PyTorch等高层框架让我们可以快速搭建复杂模型。然而，花一点时间了解下底层概念是值得的。

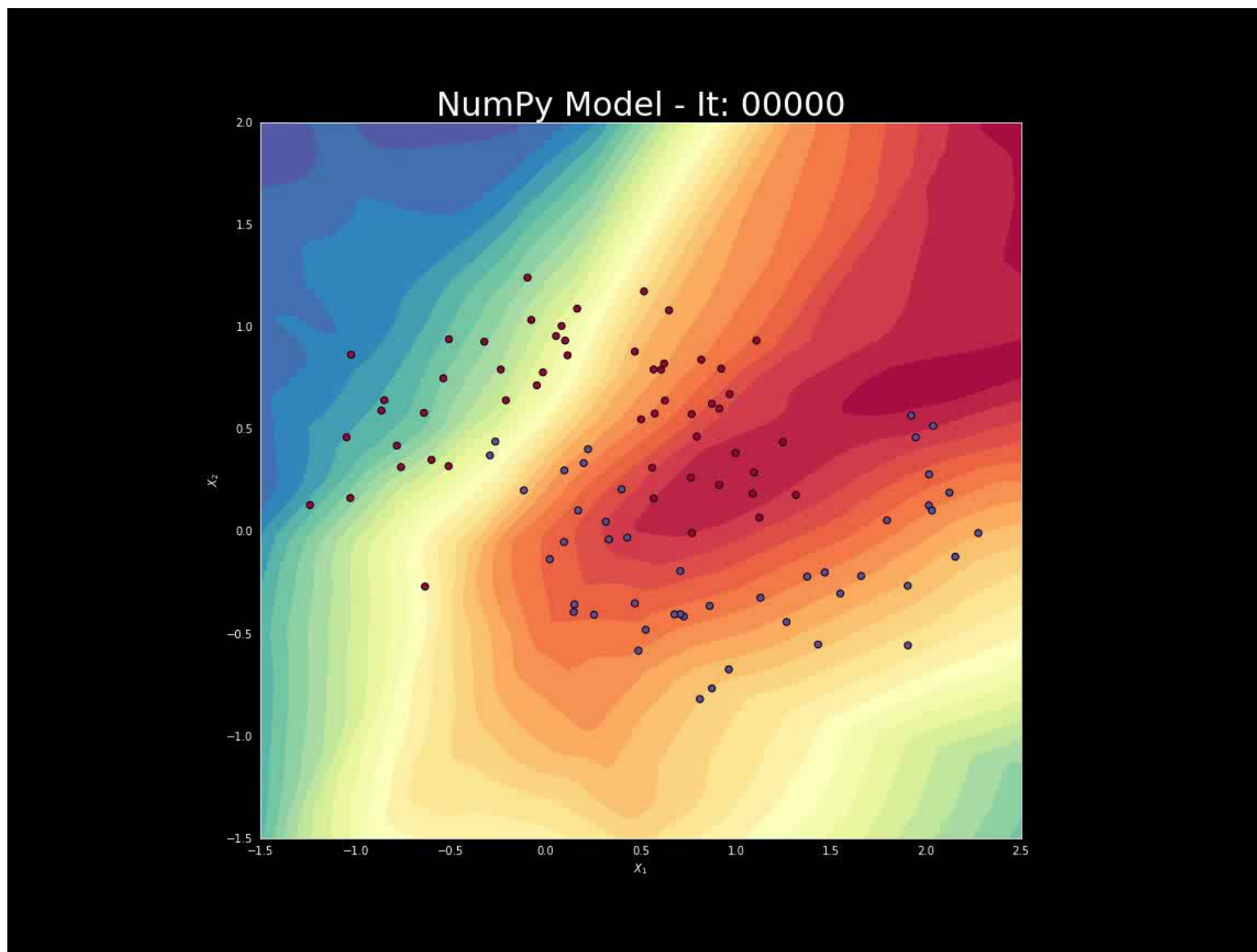


我们的目标是创建一个特定架构（层数、层大小、激活函数）的密集连接神经网络。然后训练这一神经网络并做出预测。

示意图中展示了训练神经网络时进行的操作，以及单次迭代不同阶段需要更新和读取的参数。



训练过程





Q&A

TRANSWARP
星环科技